Project Report On

Intellibot : LLM Powered academic assistant

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

in

Computer Science and Engineering

By

Nidha Rahiman Mothirapeeika (U2203159)
Noel Mathachan Mampilly (U2203169)
P S Ashna Parveen (U2203170)
Sheba Reji (U2203193)

Under the guidance of

Ms. Anu Maria Joykutty

Department of Computer Science and Engineering
Rajagiri School of Engineering & Technology (Autonomous)
(Parent University: APJ Abdul Kalam Technological University)
Rajagiri Valley, Kakkanad, Kochi, 682039
March 2026

CERTIFICATE

This is to certify that the project report entitled ***"Intellibot : LLM Powered academic assistant"*** *is a bonafide record of the work done by* ***Nidha Rahiman Mothirapeeika (U2203159), Noel Mathachan Mampilly (U2203169), P S Ashna Parveen (U2203170) and Sheba Reji (U2203193)****, submitted to the Rajagiri School of Engineering & Technology (RSET) (Autonomous) in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology (B. Tech.) in Computer Science and Engineering during the academic year 2025-2026.*

Project Guide
Ms. Anu Maria Joykutty
Asst.Professor
Dept. of CSE
RSET

Project Co-ordinator
Ms. Jisha Mary Jose
Asst.Professor
Dept. of CSE
RSET

Head Of Department
Dr. Preetha K G
Professor
Dept. of CSE
RSET

ACKNOWLEDGMENT

We wish to express our sincere gratitude towards **Rev. Dr. Jaison Paul Mulerikkal CMI**, Principal of RSET, and **Dr. Preetha K. G.**, Head of the Department and Professor of "Computer Science and Engineering" for providing us with the opportunity to undertake our project, "Intellibot : LLM Powered academic assistant".

We are highly indebted to our project coordinator, **Ms. Jisha Mary Jose**, Assistant Professor, Dept. of CSE, for her valuable support.

It is indeed our pleasure and a moment of satisfaction for us to express our sincere gratitude to our project guide **Ms. Anu Maria Joykutty** , Assistant Professor, Dept. of CSE, for her patience and all the priceless advice and wisdom she has shared with us.

Last but not the least, we would like to express our sincere gratitude towards all other teachers and friends for their continuous support and constructive ideas.

Nidha Rahiman Mothirapeeika

Noel Mathachan Mampilly

P S Ashna Parveen

Sheba Reji

Abstract

IntelliBot: LLM Powered Academic Assistant is designed as a centralized intelligent learning platform that provides syllabus-aligned, accurate, and context-aware academic assistance for computer science students. The system integrates learning materials across multiple core computer science subjects, enabling students to access reliable information through a single interactive chatbot interface.

The system is built using the Retrieval-Augmented Generation (RAG) framework, which combines curated academic content with generative AI to produce factual and syllabus-based responses. IntelliBot features a **React.js frontend interface, a FastAPI backend** for query processing, and a semantic search pipeline powered by **FAISS** for efficient document retrieval. Additionally, the platform incorporates **Firebase authentication and persistence mechanisms**, enabling subject-wise chat history storage and retrieval for personalized learning experiences.

An admin management portal allows instructors to securely upload, manage, and update course materials, ensuring content integrity and controlled access to academic resources. The system supports multi-subject interaction, enabling students to switch between subjects while maintaining context-specific responses.

This implementation demonstrates the scalability and practicality of RAG-based AI systems in educational environments. IntelliBot establishes a flexible foundation that can be expanded to cover additional subjects, ultimately evolving into a comprehensive AI-driven academic assistant for the entire computer science curriculum.

Contents

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
COA	Computer Organization and Architecture
CPU	Central Processing Unit
DBMS	Database Management System
DFD	Data Flow Diagram
FAISS	Facebook AI Similarity Search
JSON	JavaScript Object Notation
LLM	Large Language Model
NLP	Natural Language Processing
OS	Operating System
PDF	Portable Document Format
RAG	Retrieval-Augmented Generation
RAM	Random Access Memory
UI	User Interface
UML	Unified Modeling Language

List of Figures

List of Tables

Chapter 1

Introduction

Through the development of intelligent technologies that provide efficient information access for users, artificial intelligence has transformed a number of industries, including education. Students often use a range of materials, such as lecture notes, textbooks, and online resources, to understand challenging subjects in academic contexts. However, quickly locating relevant information can be challenging and time-consuming. IntelliBot is an AI-powered academic assistant that allows students to interact with syllabus-based learning resources via a chatbot interface. By combining state-of-the-art online technology with astute retrieval techniques, the system aims to provide accurate and context-aware replies to academic queries.

1.1 Background

Recent advancements in natural language processing and Large Language Models (LLMs) have enabled the development of conversational AI systems capable of understanding user queries and generating meaningful responses. These systems allow users to access information through natural language interactions, making them useful tools for knowledge assistance and educational support.

However, many general-purpose AI chat systems are not specifically aligned with academic syllabi and may produce responses that lack contextual accuracy. Retrieval-Augmented Generation (RAG) addresses this issue by combining document retrieval with generative AI models. In this approach, relevant information is first retrieved from curated course materials and then used to generate accurate responses. IntelliBot utilizes this architecture to provide syllabus-aligned academic assistance for computer science students through an interactive web-based platform.

1.2 Problem Definition

Students often face difficulty locating relevant academic information for specific subjects within their curriculum. Searching through lecture slides, textbooks, and online resources can be time-consuming and may result in inconsistent or unreliable information. Therefore, the aim of this project is to develop an intelligent academic assistant that provides syllabus-based responses by integrating curated course materials with AI-driven conversational capabilities.

1.3 Scope and Motivation

The scope of the IntelliBot system includes the development of a web-based academic chatbot that supports subject-specific question answering for computer science courses. The system integrates document retrieval mechanisms with large language models to generate contextual responses based on curated academic materials. Additionally, the platform includes an admin dashboard that allows instructors to upload and manage course materials, ensuring that responses generated by the chatbot are aligned with the syllabus.

The motivation behind developing IntelliBot is to simplify access to academic knowledge and improve the learning experience for students. By enabling students to interact with course materials through a conversational interface, the system reduces the time spent searching for information across multiple sources. IntelliBot aims to provide a centralized learning assistant that enhances academic productivity and supports effective learning.

Table 1.1: Core Components of the IntelliBot System

Component	**Technology Used**	**Purpose**
Frontend	React.js	User interface for chatbot interaction
Backend	FastAPI	Handles query processing and APIs
Retrieval Engine	FAISS	Semantic search over course documents
Database	Firebase	Authentication and chat persistence

1.4 Objectives

1. To design and develop an AI-powered academic chatbot for computer science students.
2. To integrate Retrieval-Augmented Generation (RAG) techniques for improving response accuracy.
3. To implement subject-wise document retrieval using vector embeddings.
4. To develop a user-friendly web interface for chatbot interaction.
5. To implement an admin dashboard for uploading and managing course materials.
6. To enable chat history storage and retrieval for personalized learning experiences.

1.5 Challenges

Developing an AI-powered academic assistant presents several challenges. One of the main challenges is ensuring that responses generated by the language model remain accurate and aligned with course materials. Another challenge involves efficiently retrieving relevant documents from large datasets using semantic similarity search. Integrating multiple system components such as the frontend interface, backend APIs, and AI retrieval pipeline also requires careful system design.

1.6 Assumptions

The following assumptions were considered during the development of the IntelliBot system:

- Students will access the system through a web-based platform.
- Academic content uploaded by instructors is accurate and syllabus-aligned.
- Users have access to stable internet connectivity.
- Users possess basic familiarity with chatbot-based interfaces.

1.7 Societal / Industrial Relevance

IntelliBot demonstrates how artificial intelligence can be applied to enhance educational accessibility and improve student learning experiences. By providing instant access to syllabus-aligned knowledge, the system reduces the time students spend searching for relevant learning resources. From an industrial perspective, the project highlights the practical application of AI-powered knowledge assistants in educational technology platforms. Similar systems can be adopted by universities, online learning platforms, and corporate training environments to provide intelligent learning support.

1.8 Organization of the Report

This report is organized into six chapters. Chapter 1 introduces the background, objectives, and motivation behind the IntelliBot system. Chapter 2 presents a literature survey of existing AI-based educational assistants and retrieval systems. Chapter 3 describes the system architecture, component design, and technologies used in the implementation. Chapter 4 discusses the implementation details of the system modules. Chapter 5 presents the results and performance evaluation of the developed system. Finally, Chapter 6 concludes the report and outlines possible future enhancements.

1.9 Summary of Chapter

This chapter introduced IntelliBot, an AI-powered academic assistant designed to provide syllabus-based responses for computer science students. The chapter discussed the background of AI-driven educational systems, defined the problem addressed by the project, and outlined the objectives, scope, and challenges involved in the development of the system. The structure of the report was also presented to guide readers through the subsequent chapters.

Chapter 2

Literature Survey

With the rapid growth of Artificial Intelligence in the educational sector, there has been a substantial integration of AI into education that supports improved access to education, increased efficiency in learning, and improvements in providing academic/professional support for students. AI educational systems provide students with access to automated tutoring, intelligent recommendations based upon their individual learning needs, and highly interactive learning environments supported by advanced technology. However, the majority of AI-based learning systems rely on generic data sets that are not aligned with university syllabi, making it difficult for these AI-based educational systems to provide reliable academic/professional explanations in an exam-centric way. Accordingly, there are numerous approaches to improving these types of educational assistance platforms, such as the use of Large Language Models, Retrieval and Augmentation for Generation and AI-based Tutor Systems.

2.1 AI-Based Educational Assistants

Zawacki-Richter et al. [1] has conducted extensive analysis of various AI systems within higher education (e.g., tutoring, student services, and automated assessment). The findings suggest that there is the opportunity for AI technologies to produce better learning outcomes and improve accessibility to higher education; however, the researchers focused primarily on the use of theoretical bases and do not provide any detail regarding a functional system architecture to create academic AI assistants.

The main limitation of the approach used in this review is that it did not include systems with mechanisms for syllabus-based retrieval of knowledge or academic datasets with structure. Therefore, AI systems proposed by these researchers may not be able to generate outputs that meet the requirements established through a typical university's

curricula.

2.2 Large Language Models in Education

Sharma et al. [2], evaluated a variety of personalized educational/learning systems using LLM's (large language models) to provide adaptive educational experiences with generative technology. Research suggests that large language models can enhance the ability to provide interactive educational experiences through generating explanatory information, answering questions and supporting students in their understanding of concepts.

However, these systems use only internal knowledge from the LLM without a source of knowledge controlled by an instructor. As a result, the answers provided (by the LLM) may include incorrect or unverified information in the absence of curated academic datasets. Thus, the need for integrating LLMs with structured and formalized academic knowledge bases is evident.

2.2.1 ChatGPT and Conversational AI in Education

In their study on the possible applications of conversational AI models like ChatGPT in education, Kasneci et al. [3] found many benefits associated with using AI chat systems in this environment (e.g., providing on-demand clarification to any question and supporting problem-solving or conversational tutoring).

However, one of the main challenges facing the authors is the issue of "hallucination," which occurs when AI models create erroneous or non-existent information. Since these models do not perform a verification of responses against other sources of factual data, they may provide students with faulty or unreliable academic instruction.

2.3 Retrieval-Augmented Generation Systems

In allotted time, Lewis et al. [4] proposed the use of Retrieval-Augmented Generation (RAG), which is a combined model of retrieving documents and then generating language from those documents by using an external knowledge base to retrieve documents that would be relevant for generating a response.

This generally allows for more accurate facts, as well as less hallucination of any kind due to the fact that the responses will be based upon actual documents. Because

vector-type databases (e.g., FAISS) have the ability to perform efficient semantic similarity searches to determine the similarity between an incoming user query and the stored document embeddings, this all works quite well—at least from a general RAG standpoint. However, the RAG is a task agnostic model and therefore does not have a particular item emphasis with regards to curriculum-based academic types of applications.

More recently, researchers such as Hu et al. [5] have been looking at ways to apply parameter efficient fine-tuning techniques (for example, LoRA) in conjunction with retrieval systems to improve domain specific reasoning. While this results in performance improvements to the models, these improvements primarily occur in areas such as healthcare and law; therefore, the application of fine-tuning approaches to academic curriculum based systems is yet to be widely used.

2.4 Summary and Gaps Identified

Table 2.1: Comparison of Existing Approaches

Paper	**Approach**	**Advantages**	**Limitations**
Zawacki-Richter et al.	AI in Education Review	Highlights importance of AI in higher education	No technical framework for academic chatbot
Sharma et al.	LLM-based learning systems	Interactive and personalized learning	No teacher-controlled knowledge base
Kasneci et al.	ChatGPT in education	Strong conversational explanation ability	High hallucination risk
Hu et al.	LoRA + Retrieval models	Improved domain-specific reasoning	Not applied to academic curriculum systems
Lewis et al.	Retrieval-Augmented Generation	Reduces hallucination using external knowledge	Generic framework without syllabus alignment

Through an examination of previous studies, it is possible to discern some large areas of missing research:

1. Many if not most Artificial Intelligence (AI) Chat systems use general datasets for their conversation models, rather than aligning with university syllabi.

2. Currently, there are no instructor-controlled knowledge bases in the existing Large Language Model (LLM)-based systems.

3. There are many conversational AI systems that provide false or inaccurate updates (referred to as "hallucinations") and thus are unreliable.

4. There is a lack of focus in current research on organizing subject-wise academic knowledge.

5. Very few AI systems utilize retrieval-based AI models as well as curated educational materials for use in student learning.

The IntelliBot project aims to overcome these barriers by combining a syllabus-oriented knowledge database (like a regular textbook) with a Retrieval-Augmented Generation architecture and leveraging semantic retrieval through FAISS vector data bases and asynchronous web-based subject area academic support. More importantly, this will keep chatbots in normal conversational mode based on verified course content.

2.5 Summary of Chapter

The previous chapter reviewed numerous studies regarding AI-assisted education, large language models, and retrieval-augmented generation. The review showed advantages and disadvantages of current solutions, as well as identified major gaps in open research regarding academic systems using AI aligned with syllabi. Based on those studies, it was determined that the IntelliBot will serve as a retrieval-augmented academic assistant for computer science students by providing answers that are reliable and based off their syllabus.

Chapter 3

System Design

IntelliBot's system architecture was designed for maximum scalability and modularity as an academic help agent that provides curriculum-aligned answers to students in computer science [6][7]. The application is structured using cutting-edge web technologies, AI-based retrieval techniques, and structured knowledge bases in order to guarantee accurate and reliable academic support. There is a layer of separation between user interface, application logic, knowledge retrieval pipeline, and data processing components which enforces the modular approach to improve overall maintenance, scalability of application, and efficiency of system integration.

3.1 Overall Architecture

The IntelliBot system is based on a multi-layered, component-based architecture consisting of five major components including: the User and Presentation Layer; the Application and Orchestration Layer; the RAG Processing Layer; the AI and Data Layer; and the Offline Data Preparation Layer [4][8].

1. **The User and Presentation Layer** facilitates interaction between the user (the Student and Instructor) and the system. Users can make queries, select subjects, and view responses provided by the chatbot using a web-based interface; the front end application creates API calls to the back-end server and presents the generated response(s) back to the user within that interface.

2. **The Application and Orchestration Layer** governs the overall workflow of the system by serving as a mediator between users and the system. Once the FastAPI back-end receives a user query, it executes the necessary logic to complete the user's request and communicate the results back to the user via the front end interface

as well as execute any logic required to communicate with the other components of the IntelliBot system. The Application and Orchestration Layer will also govern how user authentication is managed, and will also coordinate the communications between the front-end interface and the AI processing modules.

3. **The RAG Processing Layer** executes intelligent query processing. Once a user submits a query, the system converts it to vector embeddings and compares them against stored data in the vector database. The query is then matched with the most relevant academic content and combined with the original query into one augmented prompt. This final prompt is submitted to the Language Model to produce a response in context [4].

4. **The AI and Data Layer** consist of all the core AI models and the vector storage systems. The Language Model generates responses using the Contextualised academic data. The FAISS vector database contains all of the vector embeddings generated from the course-related documents and the syllabus [8].

5. **The Offline Data Preparation Layer** is responsible for preparing the academic knowledge base prior to run time. Course materials such as syllabus documents, notes and PDF files are processed to extract text, clean text, segment text and generate embeddings using text processing methods. The embeddings are then saved in the vector database for later use in query retrieval.

3.2 Component Design

There are 4 major components that make up the IntelliBot system of providing accurate academic answers.

The User Interface Component is the layer that allows users to interact with the system. Students will use the web interface to select subjects and ask academic questions of the chatbot, as well as see the chatbot's answers. Teachers will use the user interface to access administrative functions, like updating and managing the student's materials.

The Backend Processing Component is developed in FastAPI, which handles user requests and operates the system's logic. Additionally, the backend processing component manages communication between various modules of the IntelliBot system. Backend

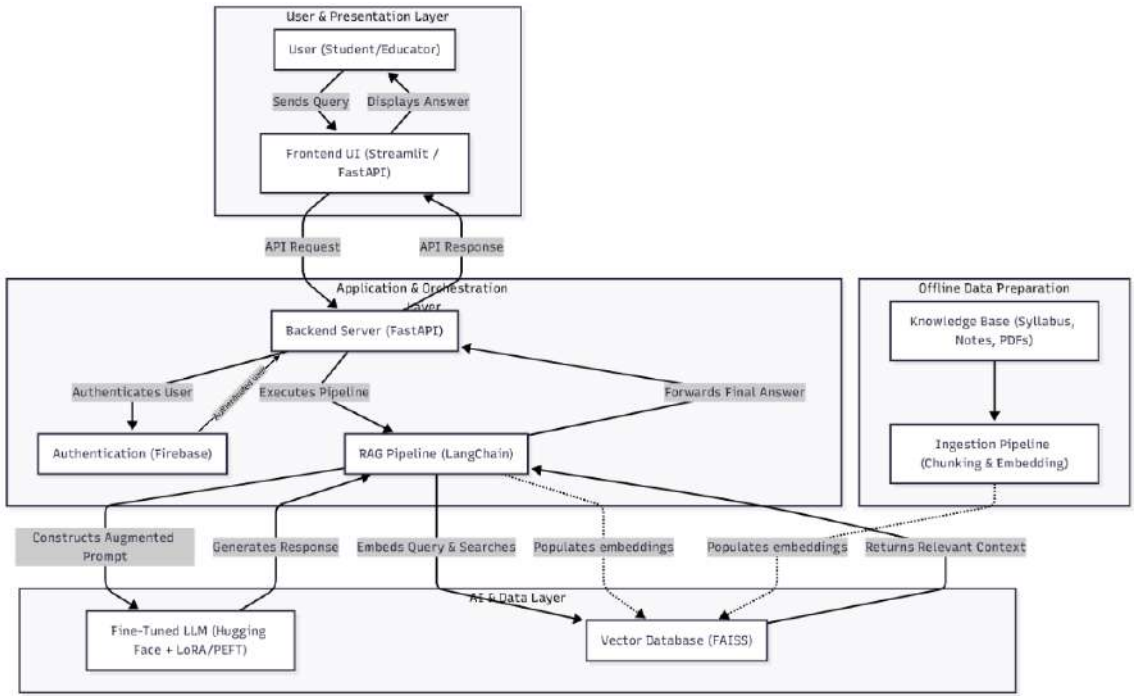


Figure 3.1: High-level architecture of the IntelliBot academic assistant system.

processing of the queries is performed using the retrieval pipeline to generate the reply to users.

The Retrieval and Query Processing Component implements the Retrieval-Augmented Generation Pipeline (RAG) using LangChain [4][8]. This component retrieves academic information from the vector database based on the user request, and then retrieves relevant information to pre-populate the language model with contextual prompts.

The AI Generation Component uses a language model that is fine-tuned with academic materials to generate a reply based on the academic material that has been retrieved from the user request. This guarantees that answers generated by the language model reflect the subject area being requested by the user, rather than being broadly representative of what the language model knows [3][9].

The Knowledge Base Component contains all the structured academic materials; syllabi, lecture notes, and textbooks are all stored within the Knowledge Base Component. Each of these documents are converted into a vector embedding and stored in the FAISS vector database to permit efficient semantic searches of the knowledge stored in the knowledge base [8].

3.2.1 UML Diagrams

Three major diagram types represent various aspects of the design of IntelliBot. They are: The Use Case Diagram, the Data Flow Diagram, and the Sequence Diagram. These diagrams illustrate user interactions with the system as well as how data travels through various parts of the architecture's design.

Use Case Diagram

A Use Case Diagram depicts the main actors of the system and their interactions with one another. The two main actors are the Admin (Teacher) and the Student. Once an Admin logs into the system they can manage the Knowledge Base by uploading and maintaining subject-related notes. Students can log into the platform, select a subject they would like to work on and view available notes. Students may also use the chatbot to get answers to questions they have regarding the subject matter. The chatbot processes the query, retrieves relevant information, and maintains the chat history for future reference.

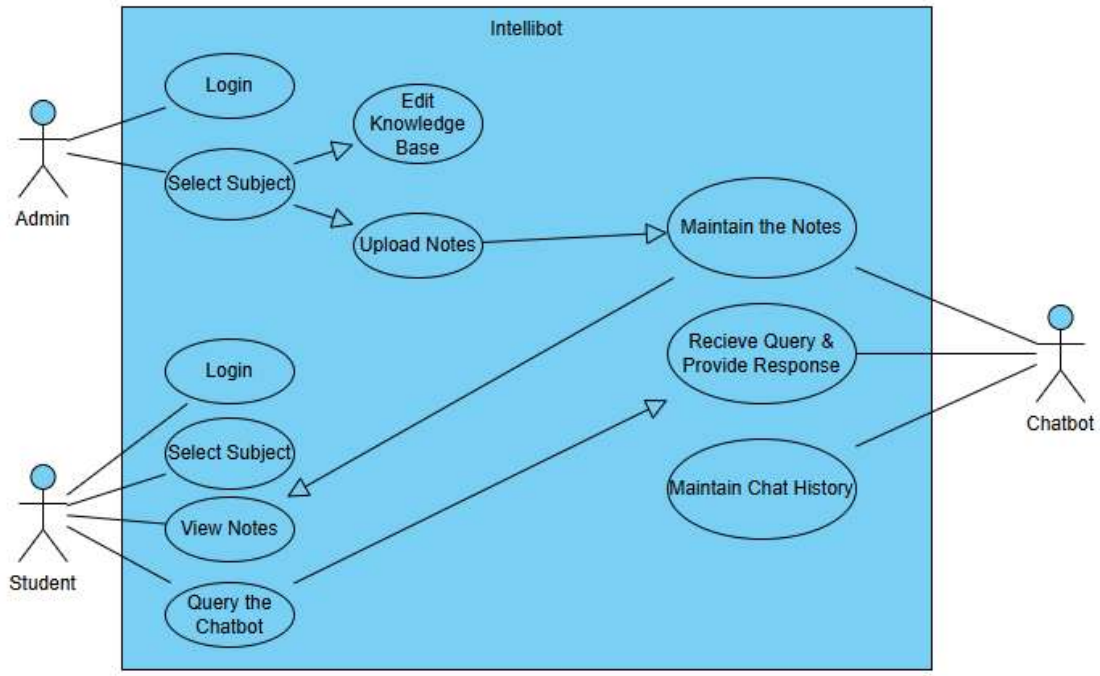


Figure 3.2: Use Case Diagram of IntelliBot System

Data Flow Diagram

The Data Flow Diagram shows how data moves through the IntelliBot system. This data flow diagram shows two major data flows: Teacher data flow and Student data flow. The Teacher data flow is represented by the Admin (Teacher) who logs into the IntelliBot system and uploads an academic document that represents the Teacher's subject area; this document can be notes or textbooks. The academic documents undergo Document Processing, Text Extraction and Text Cleaning before the academic document is transformed into an Embedding. The Embedding is indexed and stored in the FAISS Knowledge base.

Upon a student selecting a subject and submitting a question, this event triggers semantic search to discover the appropriate context from the FAISS index [8]. The FAISS index will then return the relevant context for the student's question. When the FAISS index returns this contextual information, the system will augment the original question with the provided contextual information and submit it to the language model for response generation [4]. The final response will then get displayed to the student.

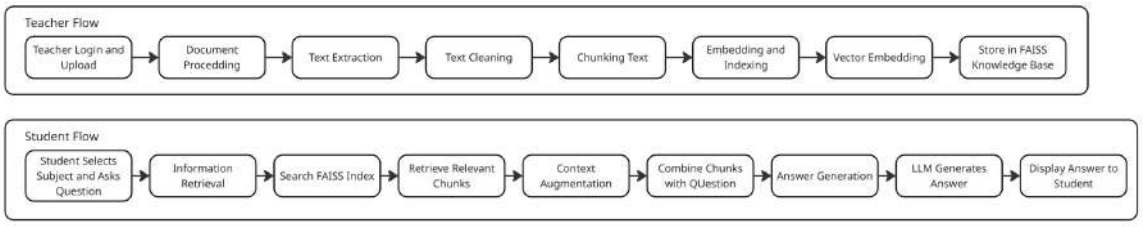


Figure 3.3: Data Flow Diagram showing teacher and student interaction pipeline

Sequence Diagram

The Sequence Diagram captures this process and the flow of events between the interacting components in the system as they relate to processing of a query. When a student submits a question through the user interface, the front-end system sends this question to the back-end API, which forwards it to the chatbot core. Upon receipt of the request from the chatbot core, the core will query the FAISS dictionary for any content (provided by admins) that is relevant to the student's question and return it to the core. After the core provides relevant content to the question, the core combines the question with relevant content and returns that combined input to the language model for generating the response [4]. The language model's generated response will then be returned to the

front-end for display to the student.

Additionally, any time an administrator uploads new notes to the system, the knowledge base is updated, enabling all queries submitted after this update to use the newly added academic content.

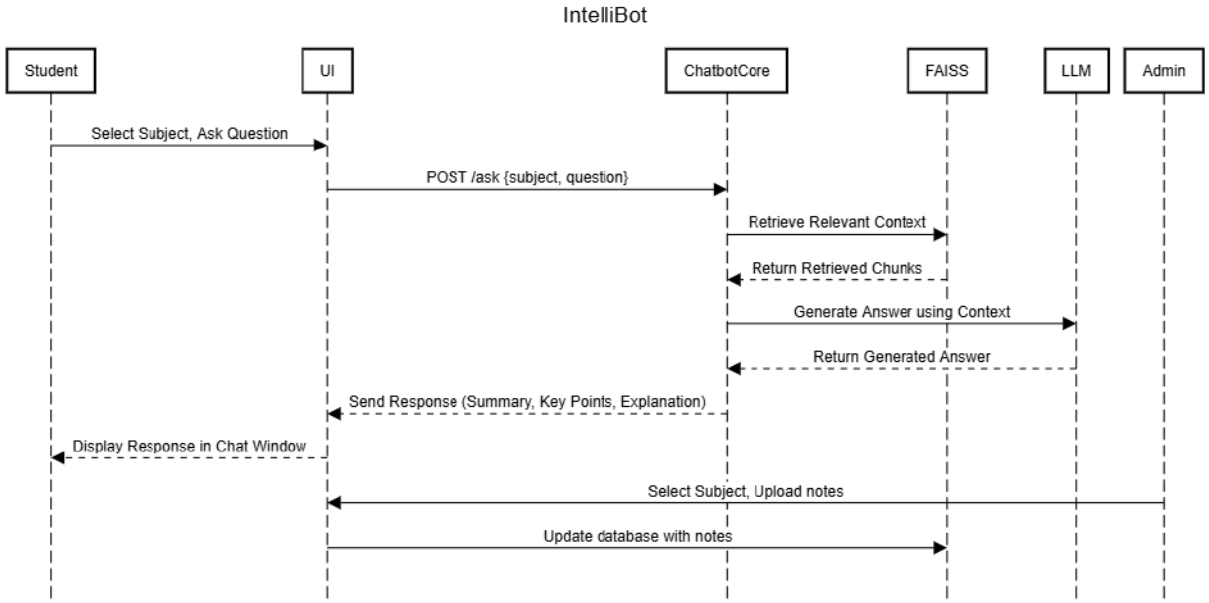


Figure 3.4: Sequence Diagram illustrating query processing in IntelliBot

3.3 Module Division

The IntelliBot is organized into several functional modules, allowing for modular development that is scalable and maintainable. Each module manages a designated function in the academic assistant system.

3.3.1 User Interface Module

The User Interface (UI) module is the interface, or interaction layer, between the users (students and administrators/teachers) and the IntelliBot. The UI module allows students to select subjects, ask questions, and see the responses generated by the IntelliBot. Additionally, the UI module allows administrators/teachers to upload and manage academic materials (notes). The UI is built using React, and interacts with the backend using API requests.

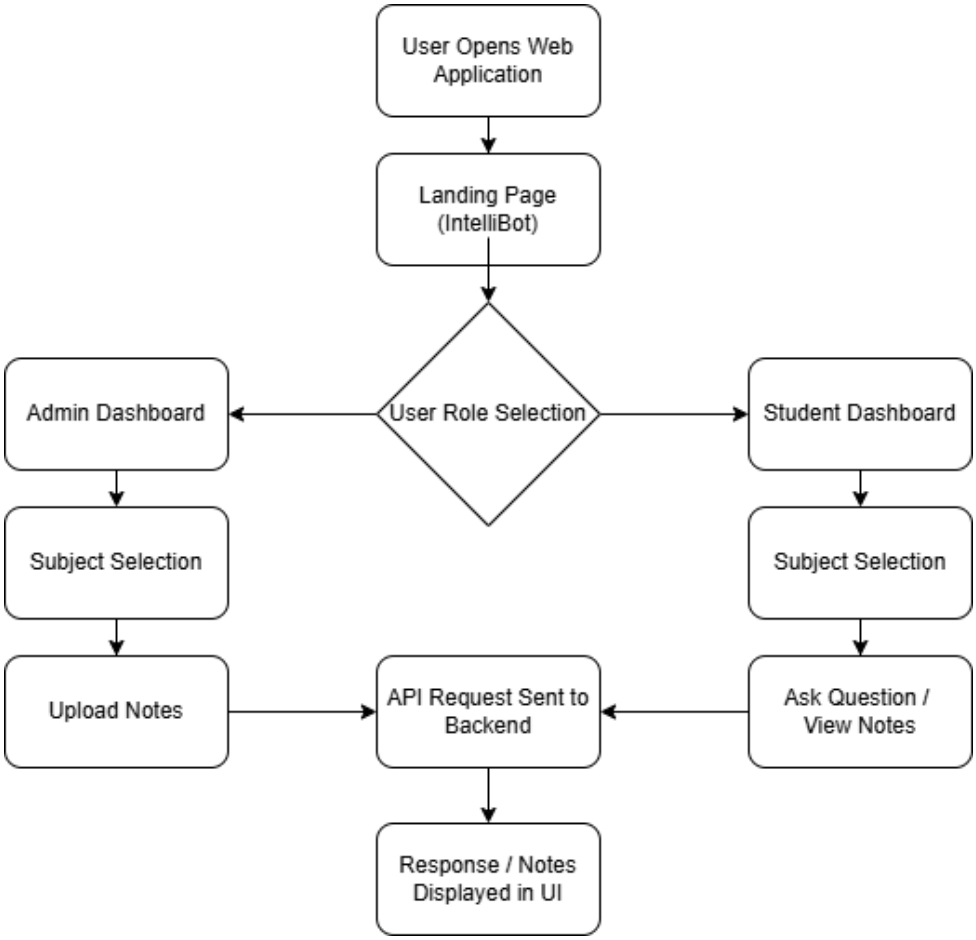


Figure 3.5: User Interface Module showing interaction between users and the IntelliBot system

3.3.2 Chatbot Core and Query Processing Module

The Chatbot Core and Query Processor module processes student queries and generates responses. When the student submits a question, the system identifies the subject selected by the student and then sends the query to the backend server. The module queries the knowledge base for relevant academic data that serves as the basis for generating an augmented prompt to send to the language model to generate a response.

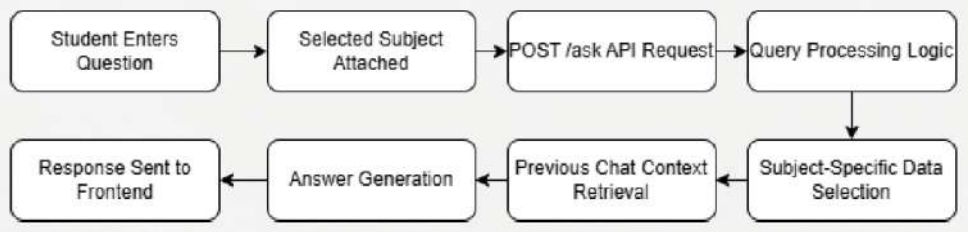


Figure 3.6: Chatbot Core and Query Processing Module

3.3.3 Knowledge Base and Data Processing Module

The Knowledge Base and Data Processing module prepares academic materials for retrieval. It processes syllabus documents, notes and textbooks into structured data. This module performs text extraction, cleaning, segmentation and generation of an embedding for each academic material processed. The resulting embeddings are stored in the FAISS vector database and used during query processing for semantic search.

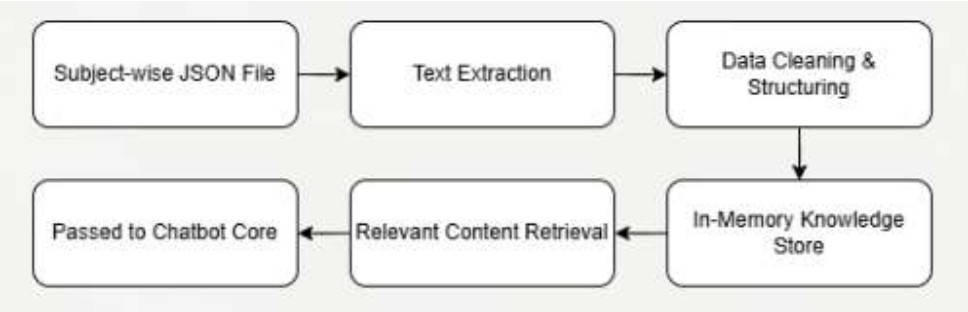


Figure 3.7: Knowledge Base and Data Processing Module

3.3.4 Notes Management Module

Utilising the Notes Management Module, instructors have the ability to upload their own course materials into a searchable database. Once uploaded, these files are automatically processed into a format that can be used by ChatGPT when answering student queries. In addition, students will be able to view uploaded files through the graphical interface of the Notes Management Module.

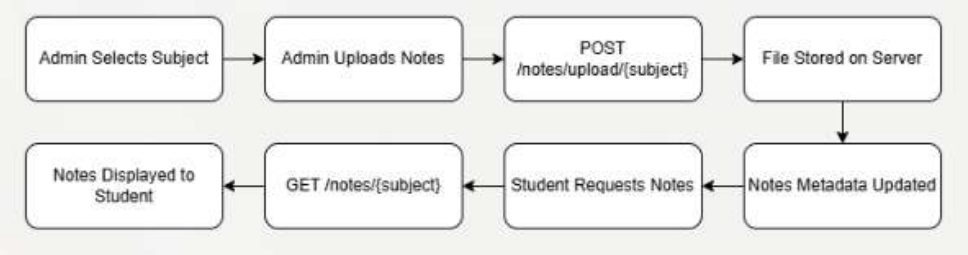


Figure 3.8: Notes Management Module

3.4 Tools and Technologies

The IntelliBot system utilizes several software tools and technologies to implement its functionality.

Hardware Requirements

- Minimum 8GB RAM
- Intel i5 processor or higher
- Internet connectivity

Software Requirements

- React for frontend development
- FastAPI for backend API services
- Python for AI processing and backend logic
- LangChain for implementing the RAG pipeline
- FAISS for vector similarity search
- Firebase for authentication and chat persistence
- HuggingFace libraries for language model integration

3.5 Work Breakdown and Responsibilities

The development of IntelliBot was shared among a number of different developers based upon the various components of the overall system.

Sheba Reji was involved in the software development and implementation of the RAG (Response Allocation via Generative) pipeline as well as designing the FastAPI back end, integrating embeddings and FAISS retrieval capabilities to ensure that responses generated by ChatGPT were aligned with syllabi.

Nidha Rahiman Mothirapeedika focused on the development of the front end for IntelliBot. This involved creating a user interface using React and building features for users to perform chat interactions with ChatGPT and connect via API to the back-end server.

P.S. Ashna Parveen worked on preparing datasets and knowledge base for developing IntelliBot. She did this by creating structured JSON files containing all academic content related to Operating Systems and auditing academic content to ensure that documents were accurate and aligned to their syllabus.

Noel Mathachan Mampilly was responsible for design documents for the overall system architecture, coordinating the integration of modules into the overall IntelliBot system, preparing project documentation, and assisting with the testing of the IntelliBot system.

3.6 Key Deliverables

The IntelliBot project produced several key deliverables.

- A fully functional AI-powered academic chatbot capable of answering syllabus-based questions.
- A web-based interface for students to interact with the chatbot.
- An administrative portal for instructors to upload and manage academic notes.
- A retrieval-based knowledge system using FAISS vector databases.
- Persistent chat history storage and subject-wise chat management.

3.7 Project Timeline

IntelliBot's development was structured in phases with defined timelines. The first was project management/literature review followed by backend development including cre-

ation of knowledge base; then frontend and system integration and the last was testing/debugging/improving performance and preparing for project evaluation.

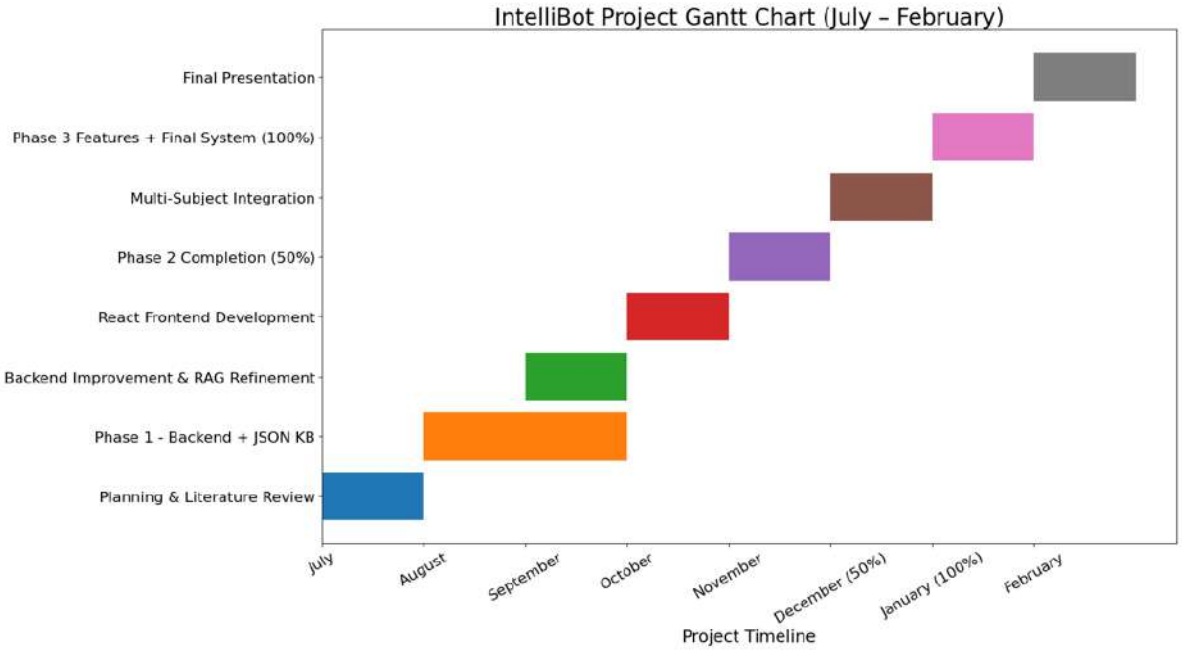


Figure 3.9: Project timeline of IntelliBot development phases.

3.8 Summary of Chapter

This chapter describes IntelliBot's design structure, which employs a layered architecture, outlines the principal areas and modules, as well as describes the technology used for building out these areas and modules. This chapter also outlined who is responsible for all of the areas of responsibility and what key deliverables were created in this project development process. The modular design offers a way for the ability to expand the system into additional subjects or academic domains in the future and is thus an excellent foundation for developing the expansion of IntelliBot

Chapter 4

Methodology and System Implementation

This chapter explains the methodology that has been utilized to design and implement the IntelliBot academic assistant system. The methodology is based on a Retrieval-Augmented Generation (RAG) framework to generate answers to student queries that are related to their syllabus. The methodology is designed to incorporate document processing, semantic search, and language model generation to guarantee accurate and reliable academic explanations. The architecture of the system is designed to incorporate a React frontend, a FastAPI backend, and a FAISS vector database to guarantee efficient information retrieval. The academic assistant system is designed to utilize the power of technology to generate answers from verified academic sources instead of relying solely on language model knowledge. Academic assistant systems have gained significant importance in modern education systems that utilize the power of artificial intelligence to assist students in understanding complex topics and retrieving academic knowledge [1][3][7].

4.1 System Workflow

The general workflow of the entire system can be described as follows. The workflow of the entire system begins when a student chooses to interact with the chatbot. The student has first to choose a subject and enter a question, which is then forwarded to the server as an API call. The server processes the query, which is then forwarded to the Retrieval-Augmented Generation model. The RAG model converts the query into vector form, which is then compared using a similarity search against a vector database. The most relevant academic content is then retrieved and added to the query, which is then forwarded to the language model. The response is then generated based on the query and the academic content, which is then forwarded as a response to the student.

This workflow is important because it ensures that the chatbot provides information

based on the syllabus, not on the knowledge of the model. The use of a retrieval system has been proven to be more reliable in providing factual information compared to using only a model, as it prevents hallucination in AI-generated content [9].

4.2 Retrieval-Augmented Generation Methodology

The IntelliBot system uses the "Retrieval-Augmented Generation" (RAG) architecture. The "RAG" method uses information retrieval techniques in combination with a "generative" language model to increase the accuracy of the responses. Instead of generating responses with the help of a pre-trained "language model," the IntelliBot uses the "RAG" method to retrieve relevant information from a "knowledge base."

The "RAG" method has proved to be an "effective solution" to "knowledge-intensive tasks" with the help of "large language models" [4]. The "RAG" method has allowed the model to "utilize external knowledge" while generating responses. The recent surveys on "RAG" models have shown the effectiveness of this model in reducing "hallucination" in AI responses [8].

In the case of the IntelliBot model, the "RAG" method retrieves subject-specific "academic content" from a "vector database" to generate responses. The responses are generated in a manner that they are "grounded" in "syllabus-based academic materials" instead of "general knowledge."

4.3 Knowledge Base Construction

The knowledge base acts as a repository for academic knowledge used by IntelliBot. Academic sources such as lecture notes, books, and syllabus documents are collected from teachers via the administrative portal.

After uploading the documents, document preprocessing occurs to prepare the data for semantic retrieval. In this step, text extraction and removal of unwanted formatting details are done to clean the raw data. The cleaned data is then divided into smaller parts to optimize the efficiency of vector search operations.

A numerical vector is created for each part of the text data using an embedding model. Embedding models are used to extract semantic meaning from text data and perform similarity queries to compare user queries with existing content. Transformer-

based models and parameter-efficient fine-tuning methods like LoRA have demonstrated substantial benefits in domain-specific reasoning in large language models [5][10][11].

The created text data embeddings are then stored in a FAISS vector database.

4.4 Embedding and Vector Retrieval

The semantic search is carried out using the FAISS vector database. The Facebook AI Similarity Search is a library for efficient similarity search of the nearest neighbors in high-dimensional vector spaces. This enables the system to retrieve the most relevant document segments according to their semantic similarity to the query embedding.

When a query is entered by the user of the system, the system generates an embedding of the query using the same embedding model as that of the document dataset. The system carries out a similarity search using the FAISS database, resulting in the most relevant document segments. The segments of the document, along with the query entered by the user of the system, are considered for generating the response using the language model.

The method of retrieving the answers from the system ensures that they are always from academic materials, reducing the chances of incorrect information being provided.

4.4.1 Query Processing and Response Generation

The query processing module is responsible for the interaction with the frontend interface, backend server, retrieval system, and language model.

When a query is given by the student, the system first determines the selected subject context. Then, the backend server sends the query to the retrieval system to obtain the relevant knowledge segments.

Once the contextual information is received from the retrieval system, an enhanced prompt is created by combining the query and knowledge segments in academics. This enhanced prompt is then sent to the language model to obtain a response based on the contextual information.

The response may contain explanations, descriptions, and concepts about the query. This response is then sent back to the frontend interface to display the response to the student. This is similar to knowledge-grounded conversational systems to assist students in their studies [6][7].

4.5 Chat Memory and Persistence

To provide a better learning experience for the user, IntelliBot provides a memory store for the chat. The memory store is provided through the use of Firebase to store the user's chat conversations. This helps the student to access their previous conversations.

The memory store for the chat provides dynamic management of the chat. The user can start a new conversation, change the name of the chat according to the initial query, delete the conversation, and resume their previous conversations.

4.6 Key Features of the IntelliBot System

There are various important features in the IntelliBot system, which make it accessible and convenient in terms of academic support.

The first feature is subject-wise knowledge retrieval. The academic resources are categorized according to subjects like **"Operating Systems," "Database Management Systems," "Computer Organization & Architecture," "Python Programming," "Compiler Design,"** etc.

The second feature is the instructor-controlled knowledge base. The teachers are allowed to upload the lecture notes or academic resources by making use of the administrative panel. The chatbot responses are generated from reliable academic resources.

Moreover, the IntelliBot system offers dynamic chat management, which enables the students to manage the chat effectively.

4.7 System Integration

The IntelliBot system utilizes different technologies to create a complete system for an academic assistant. The frontend of the system utilizes React, which creates an interactive interface for the system. The backend of the system utilizes FastAPI, which handles API requests, query, and system coordination.

The retrieval pipeline utilizes LangChain, which handles prompt building and retrieval. The system utilizes FAISS, which creates a vector database for storing documents. The system utilizes Firebase, which handles user authentication and chat.

Utilizing different technologies creates a complete system that can handle different

subjects, which can further be extended to different domains in academics.

4.8 Summary of Chapter

In this chapter, the methodology that was utilized in the design and development of the IntelliBot academic assistant system has been presented. The system combines Retrieval-Augmented Generation with a structured knowledge base to facilitate accurate and syllabus-aligned answers to student queries. The chapter has highlighted the workflow of the system, the knowledge base construction process, the vector retrieval method, the persistence of the chat feature of the system, and the integration of technology. All these aspects enable the IntelliBot to function as an intelligent academic assistant to the student.

Chapter 5

Results and Discussions

5.1 Overview

This chapter presents a detailed examination of the outcomes associated with the design and evaluation of IntelliBot, an LLM-powered academic training system. The evaluation is concerned with the assessment of the Retrieval-Augmented Generation (RAG) design used within the IntelliBot system to generate syllabus-compliant academic responses that respond to the students' academic queries.

The evaluation's goal is to determine how well IntelliBot retrieves appropriate academic material from its knowledge base in order to provide contextualized and appropriate explanations to academic queries submitted by students through the academic training system's interactive chatbot interface. Academic materials that have been uploaded by academic course instructors include lecture notes, textbooks, and syllabus documents. These academic materials are processed using a document preparation pipeline that consists of text extraction, segmentation, and embedding generation, after which the generated embeddings are stored in a FAISS vector database, which facilitates semantic retrieval during query processing.

When the student submits the query, the system transforms the inputted content into vector embeddings (so-called "vector representations") and uses them to perform a semantic similarity search within the knowledge-base where all other vector representations of documents have been stored. The segments of relevant documents retrieved are added back to the initial query and become an augmented prompt, which can then be used by the language model to produce a contextually appropriate output.

The objective of this chapter is to show how IntelliBot processes queries related to computations by retrieving relevant syllabus-based documents and generating structured explanations of those documents, and includes results of evaluations of interactions be-

tween the chatbot and users (through analytics), corresponding knowledge base output, and various screenshots displaying the visual representation of the system in action. Additionally, an analysis of the performance of IntelliBot with respect to accuracy of response, retrieved content from the knowledge bases, and multi-resource integration capabilities of the system will also be performed in order to demonstrate the effective usability of IntelliBot to help with academic learning.

5.2 Results

The following figures illustrate the responses generated from the IntelliBot system based on varying academic questions submitted via the chatbot interface. The testing of this system was completed across several academic subjects such as Operating Systems, Database Management Systems, Computer Organization and Architecture, Python Programming, and Compiler Design.

When a student submits an academic question through the chatbot interface, the IntelliBot performs semantic retrieval from the FAISS vector database. Semantic retrieval involves comparing the query vector to all of the document vectors within the database to determine which of the documents contain the most relevant content to the submitted question. The document segments returned are then sent to the language model as part of the context of the response in generating a contextually appropriate response to the user's academic question.

Consequently, the IntelliBot generates structured answers to academic queries containing conceptual clarification, important points, and summaries relative to the topic you are researching. Every answer that you receive from the IntelliBot will come from a syllabus-based material and contains factual and reliable information.

The screenshots below demonstrate how the IntelliBot chatbot interacts with user requests and generates responses to inquiries about academic subjects. The results showcase the IntelliBot's ability to retrieve relevant learning materials and provide simple explanations as they relate to your course syllabus. The IntelliBot's combination of retrieval context and linguistic model generation provides students with reliable academic support via a conversational interface.

5.3 Output

In this section we will show the output interfaces of the IntelliBot system which demonstrate how the academic assistant platform functions. The screenshots below are examples of a user interacting with IntelliBot via its web-based interface.

5.3.1 Admin Interface

Figure 5.1 shows the landing page of the IntelliBot system where the user can select whether to continue as an administrator or as a student. This interface acts as the entry point of the system and allows role-based access to different system functionalities.

Figure 5.1: IntelliBot Landing Page Showing Admin and Student Access

Figure 5.2 illustrates the administrator login portal. This secure interface allows authorized instructors to access the administrative dashboard where they can manage academic content within the system.

Figure 5.3 presents the admin dashboard interface. Through this dashboard, instructors can upload or edit academic materials such as lecture notes for different computer science subjects including Compiler Design, Computer Organization and Architecture, and Database Management Systems.

Figure 5.4 shows the update notes interface where instructors can upload new course materials. The system allows teachers to select the subject, upload PDF documents,

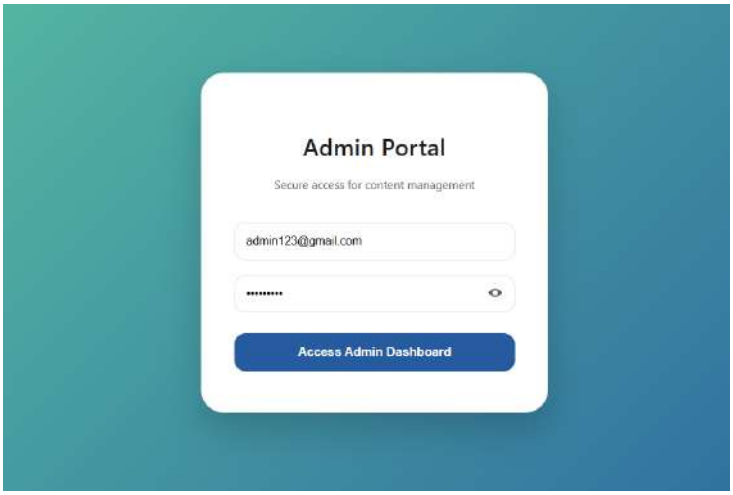


Figure 5.2: Admin Login Portal

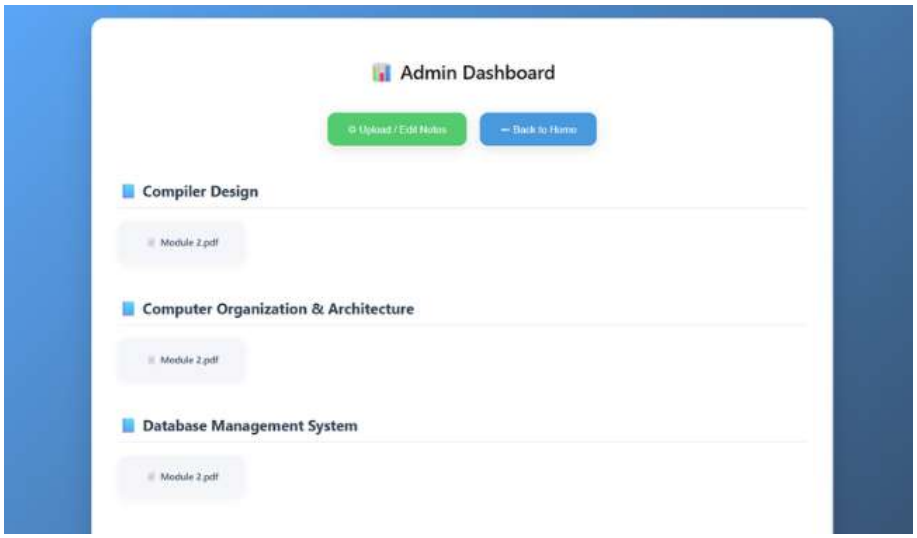


Figure 5.3: Admin Dashboard for Managing Academic Materials

and manage existing resources that will be used by the IntelliBot system during query retrieval.

5.3.2 Student Interface

Figure 5.5 illustrates the student login interface where users can securely authenticate using Google authentication through Firebase. This ensures that students can access the system and interact with the chatbot while maintaining user session data.

Figure 5.6 shows the subject selection interface available to students after logging into the system. The platform supports multiple computer science subjects such as Compiler Design, Computer Organization and Architecture, Database Management Systems,

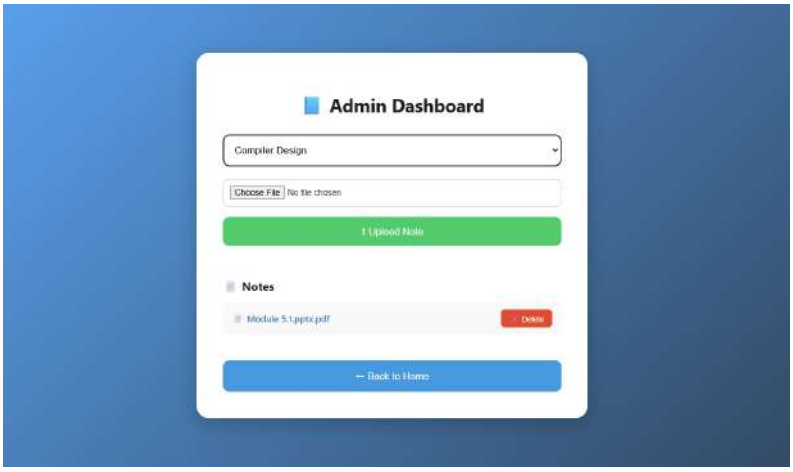


Figure 5.4: Admin Interface for Uploading and Managing Notes

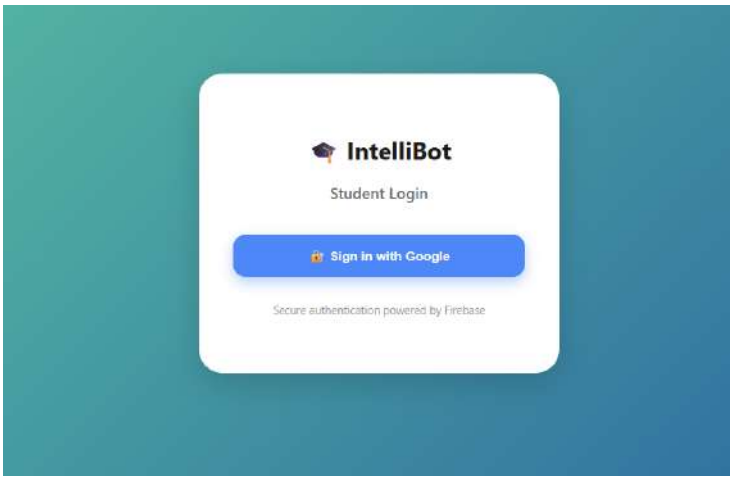


Figure 5.5: Student Login Interface

Operating Systems, and Python.

Figure 5.7 presents the chatbot interaction interface. Students can submit questions related to the selected subject, and the system processes the query using the Retrieval-Augmented Generation pipeline to retrieve relevant academic content from the knowledge base.

Figure 5.8 illustrates the response generation interface where the system displays answers generated by the chatbot. The response includes explanations based on syllabus-aligned academic materials retrieved from the FAISS vector database.

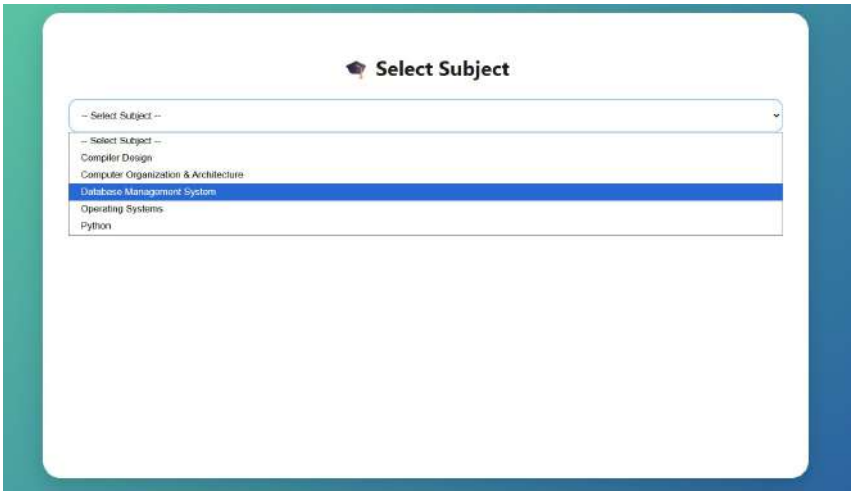
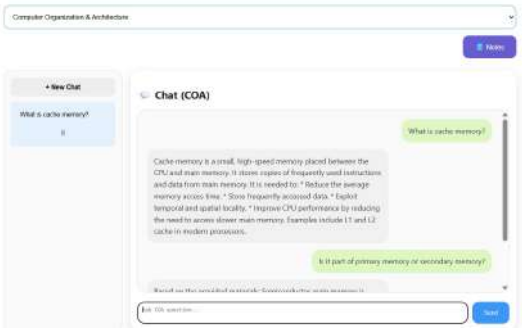
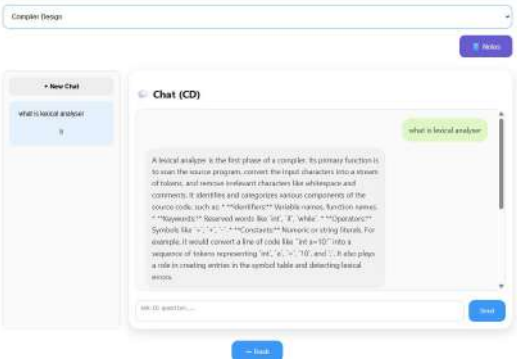


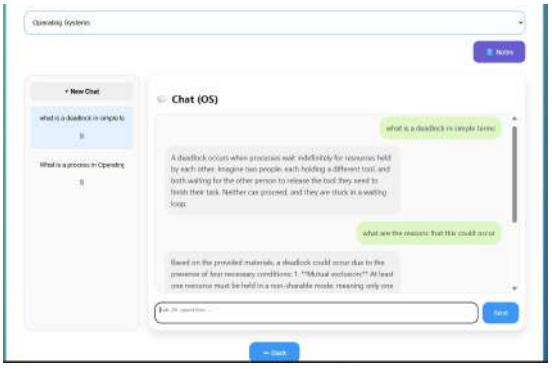
Figure 5.6: Subject Selection Interface



(a) COA chatbot interface



(b) CD chatbot interface



(c) OS chatbot interface

Figure 5.8: Generated Responses for Student Academic Queries in the IntelliBot Chatbot Interface

Figure 5.9 shows the notes viewing interface where students can access uploaded academic documents. This allows learners to directly view course materials provided by

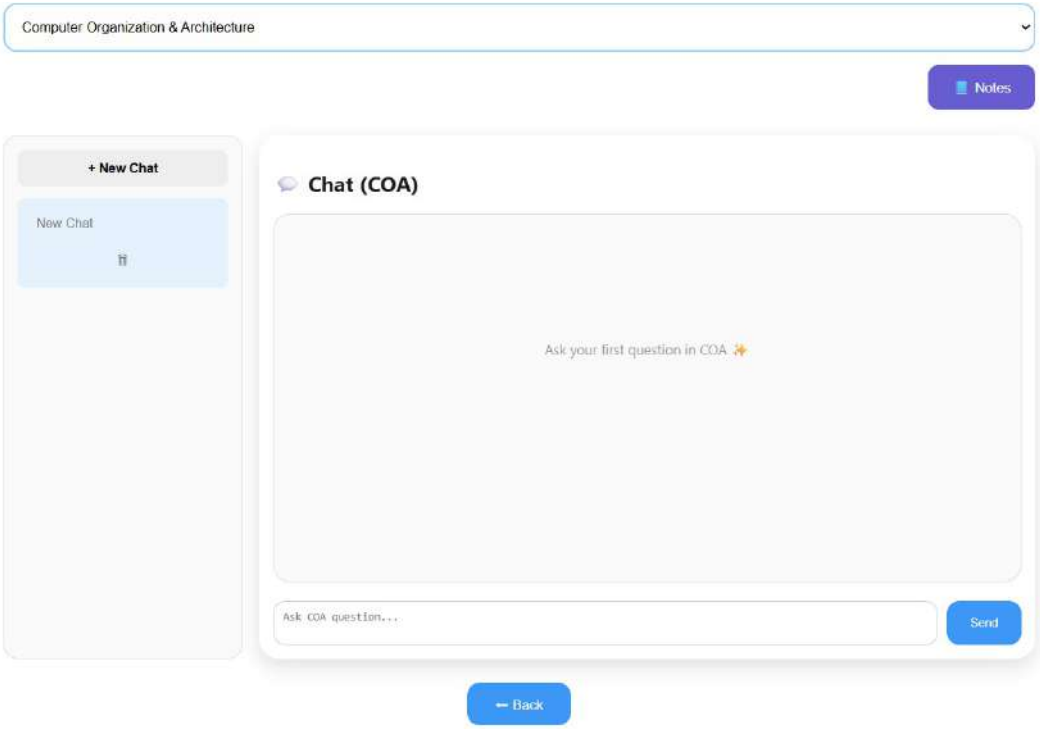


Figure 5.7: Chatbot Interface for Academic Query Interaction

instructors, ensuring that the system functions as both a chatbot assistant and a centralized academic resource platform.

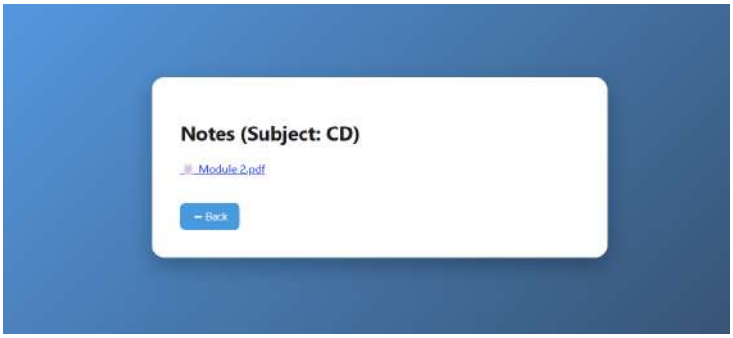


Figure 5.9: Student Interface for Viewing Uploaded Notes

These output interfaces demonstrate the practical functionality of IntelliBot and highlight how the system integrates user interaction, document retrieval, and AI-based response generation into a unified academic assistance platform.

5.4 Discussion

The evaluation results show that the IntelliBot system's use of semantic document retrieval and generative language models allows for syllabus-driven academic support. Additionally, the Retrieval-Augmented Generation architecture means that responses from the system are based on original academic sources rather than purely from the language model itself.

The system's teacher-controlled knowledge base is one of its most significant strengths. Teachers can upload curated academic content, making sure that the chatbot has access to materials rated as acceptable and appropriate by faculty. This method of utilizing the synchronous Learning module leads to more dependable and appropriate responses and consistent answers with the syllabi provided by UBC.

A significant benefit of the IntelliBot system is its organization of content by type (subject). When academic resources are catalogued by their related computer science subject areas, the retrieval system finds matching information much better. This allows for improved generated responses from the system and provides students with more specific information based on their course.

Another great feature of the system is that it maintains a log of past conversations. So students can easily go back and read any of their old conversations with the IntelliBot. This will not only facilitate the student's ability to learn, but it will also give them a way to refer back to the explanations provided by the IntelliBot when preparing for a test or assignment.

The IntelliBot also has several limitations. First, the quality of the answer that is generated by the IntelliBot is directly proportional to the amount and quality of the academic materials uploaded into the knowledge base. In the event that the knowledge base contains insufficient or unhelpful information associated with a certain question, then the answer that is given will be limited in scope. There are also a limited number of subjects that are currently available to be answered by the IntelliBot; these subjects can be expanded in the future.

5.5 Summary

In this chapter, the implementation and evaluation results for the IntelliBot academic assistant system are presented. The evaluation was successful, as it proved that the academic assistant can find corresponding academic information through a FAISS-based semantic search and can generate contextually relevant responses with the use of a Retrieval-Augmented Generation framework.

The evaluation results also indicate that the IntelliBot can provide dependable syllabus-based explanations via an interactive chatbot interface, thereby improving student access to academic knowledge. In addition, the success of this implementation has provided a solid platform for additional development of AI-enabled educational platforms with broader subject areas as well as additional learning resources.

Chapter 6

Conclusions & Future Scope

As an intelligent academic assistant designed to provide syllabus-specific and context-specific support to computer science students, the IntelliBot system integrates current artificial intelligence technologies with curated academic resources to give accurate and reliable answers to users via a conversational chatbot interface [1, 3]. The IntelliBot uses a Retrieval-Augmented Generation (RAG) approach to combine semantic document retrieval with response generation based on large language models; thus, the answers are based on verified academic content [4, 8]. The architecture of the IntelliBot system consists of a React.js frontend for user interaction, a FastAPI backend for efficient processing of requests from users, and FAISS for fast retrieval of documents based on their vector representations. The IntelliBot uses Firebase to authenticate users and provide secure access to the system as well as to manage chat history by subject; in addition, an administrator management portal allows instructors to upload and manage academic resources.

In general, the project illustrates how artificial intelligence and large language models are increasing their involvement in today's educational systems [12]. By allowing students to have one location to search for information related to their studies, IntelliBot improves the ability of students to learn efficiently by reducing the time and effort wasted searching through multiple sources. This project also shows that retrieval-based AI systems provide improved reliability for generative AI systems when they are used with domain-specific knowledge bases [9]. The use of AI-based educational assistants is a positive development for intelligent tutoring and knowledge support systems [6, 7].

Improvements that could be made to IntelliBot in the future include increasing the breadth of topics covered in the knowledge base, so it covers more computer science topics, as well as integrating new types of learning aids into the platform, such as lecture slides, textbooks and research papers. In addition, the ability to use multimodal information will be expanded within the platform to allow for the interpretation of charts/graphs, code

samples, and technical drawings. There is also an opportunity to implement personalized learning recommendations based on how users engage with the bot [2]. A central component to enable these kinds of improvements is using cloud-based technologies which allow for increased accessibility and allow for more users to engage with the platform. Finally, improvements made to retrieval techniques (for example hybrid search and re-ranking) can help improve the level of accuracy and relevance of the responses provided by the bot [8].

References

[1] O. Zawacki-Richter, V. I. Marín, M. Bond, and F. Gouverneur, "Systematic review of research on artificial intelligence applications in higher education," *International Journal of Educational Technology in Higher Education*, vol. 16, no. 39, pp. 1–27, 2019.

[2] A. Sharma, R. Kumar, and S. Patel, "Applications of large language models in personalized learning: A survey," *Computers & Education: Artificial Intelligence*, vol. 6, 100200, 2025.

[3] S. Kasneci *et al.*, "ChatGPT for good? On opportunities and challenges of large language models for education," *Learning and Individual Differences*, vol. 103, 102274, 2023.

[4] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.

[5] E. J. Hu *et al.*, "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022.

[6] J. Swacha and J. Gracel, "Educational chatbots based on retrieval-augmented generation," *Education and Information Technologies*, vol. 30, no. 2, pp. 1–23, 2025.

[7] Y. Liu, K. Chen, and J. Zhao, "Knowledge-grounded conversational agents for education," *IEEE Transactions on Learning Technologies*, vol. 17, no. 1, pp. 45–58, 2024.

[8] T. Gao *et al.*, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2023.

[9] Z. Ji *et al.*, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, Art. no. 248, pp. 1–38, 2023.

[10] L. Wang *et al.*, "Parameter-efficient fine-tuning in large language models," *Artificial Intelligence Review*, vol. 58, Art. no. 227, pp. 1–64, 2025.

[11] Y. Song *et al.*, "Efficient fine-tuning of LLaMA models using LoRA," *Expert Systems with Applications*, vol. 232, 120547, 2025.

[12] J. Qadir, "Engineering education in the era of ChatGPT: Promise and pitfalls," *IEEE Access*, vol. 11, pp. 1–14, 2023.

[13] J. Xu, Y. Li, and Z. Wang, "A survey on parameter-efficient fine-tuning of large language models," *arXiv preprint arXiv:2303.15647*, 2025.

[14] A. Gilson *et al.*, "How does ChatGPT perform on medical licensing exams?," *JMIR Medical Education*, vol. 9, e45312, 2023.

[15] T. H. Kung *et al.*, "Performance of ChatGPT on USMLE," *PLOS Digital Health*, vol. 2, no. 2, e0000198, 2023.

[16] Z. He *et al.*, "Enhancing domain-specific reasoning using LoRA and retrieval," *IEEE Access*, vol. 13, pp. 1–12, 2025.

Appendix A: Presentation

Intellibot:
LLM-Powered Academic Assistant

100% EVALUATION

Date: 03/03/2026

Project Guide: Ms. Anu Maria Joykutty

Team Members:
Nidha Rahiman Mothirapeedika - U2203159
Noel Mathachan Mampilly - U2203169
P S Ashna Parveen - U2203170
Sheba Reji - U2203193

—

Contents:

Problem Definition

Computer science students frequently depend on generic AI tools and scattered online resources that are not aligned with university syllabi and may produce inaccurate or incomplete explanations. These platforms also lack proper subject-wise organization and do not integrate teacher-verified academic materials. Consequently, students struggle to access reliable, structured, and exam-oriented content from a single trusted source, which negatively affects effective understanding and academic preparation across core subjects.

Project Objectives

1. To develop an AI-based academic assistant chatbot
2. To provide syllabus-aligned explanations for CSE subjects
3. To enable subject-wise curated knowledge retrieval
4. To integrate teacher-uploaded academic materials
5. To design scalable architecture for multi-subject expansion

Purpose and Need

Purpose

- **Centralized academic learning platform**
- **Reliable and syllabus-controlled knowledge source**
- **Assist students in exam preparation**

Need

- **Generic LLMs lack syllabus control**
- **Students need verified academic content**
- **Teachers require a platform to distribute materials**
- **AI assistants must reduce hallucination risk**

Literature Survey

Paper 1 : AI in Higher Education [1]

-Zawacki-Richter et al.

- Method: Systematic review of AI applications in higher education
- Advantage: Identifies pedagogical potential and role of AI in learning
- Limitation: Lacks technical framework for implementing academic AI assistants
- Research Gap: No architecture for syllabus-controlled academic chatbot
- How IntelliBot Improves:
  - IntelliBot provides a concrete RAG-based architecture with curated syllabus-aligned knowledge bases and subject isolation, translating theoretical AI-in-education concepts into an implementable academic assistant system.

Paper 2 — LLMs for Personalized Learning [2]

-Sharma et al.

- Method: Review of 55 LLM-based personalized learning systems
- Advantage: Demonstrates adaptive and interactive learning using LLMs
- Limitation: Absence of instructor-controlled knowledge sources
- Research Gap: No mechanism for teacher-governed academic content
- How IntelliBot Improves:
  - IntelliBot integrates a teacher-managed knowledge base and notes upload module, ensuring that all chatbot responses originate from verified academic materials rather than uncontrolled model knowledge.

Paper 3 — ChatGPT in Education [3]

-Kasneci et al.

- Method: Conceptual analysis of LLM use in education
- Advantage: Highlights conversational tutoring and explanation ability
- Limitation: High hallucination and factual inconsistency risk
- Research Gap: No grounding or verification mechanism
- How IntelliBot Improves:
  - IntelliBot uses Retrieval-Augmented Generation with FAISS retrieval to ground responses in curated syllabus documents, significantly reducing hallucinations and improving factual reliability.

Paper 4 — LoRA + RAG Hybrid Model [4]

-He et al.

- Method: Parameter-efficient fine-tuning with retrieval augmentation
- Advantage: Improves domain-specific reasoning accuracy
- Limitation: Evaluated only in healthcare and law domains
- Research Gap: No adaptation to academic curriculum environments
- How IntelliBot Improves:
  - IntelliBot adapts retrieval-augmented architecture specifically for academic syllabi, including subject-wise isolation, course-structured datasets, and exam-oriented explanations tailored for engineering education.

Paper 5 — Retrieval-Augmented Generation (RAG) [5]

-Lewis et al.

- Method: Hybrid retrieval + generation architecture
- Advantage: Enables external knowledge grounding and updatable memory
- Limitation: Task-agnostic; lacks domain governance and content curation
- Research Gap: No curriculum structuring or subject boundaries
- How IntelliBot Improves:
  - IntelliBot extends generic RAG into a syllabus-aware academic RAG system with subject-segmented vector databases, curriculum-aligned JSON schemas, and teacher-curated knowledge ingestion pipelines.

Proposed Method

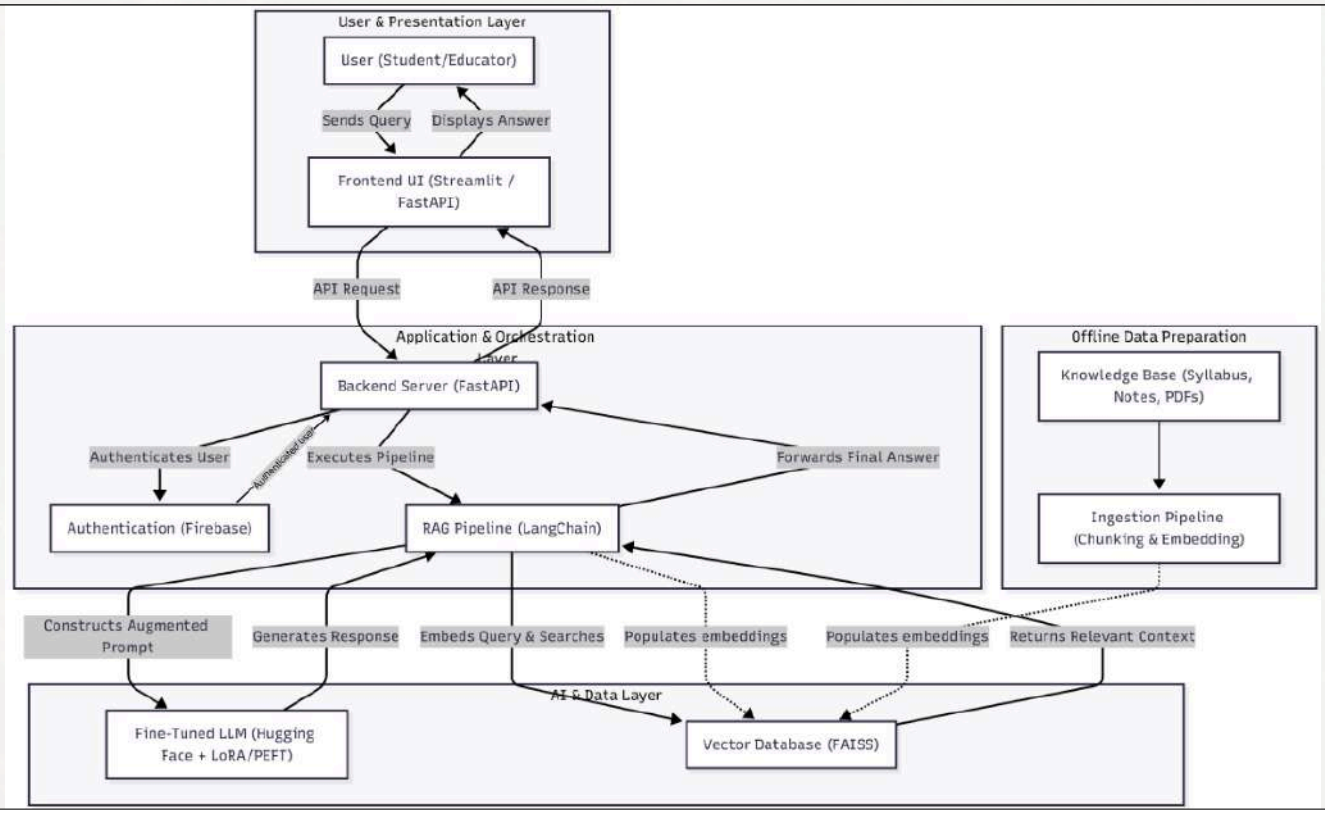
IntelliBot uses a Retrieval-Augmented Generation (RAG) architecture:

1. **Student selects subject and asks question**
2. **Query sent to FastAPI backend**
3. **Relevant content retrieved from FAISS vector DB**
4. **Context passed to fine-tuned LLM**
5. **Syllabus-aligned answer generated**
6. **Response displayed in UI**

This ensures:

- **factual accuracy**
- **syllabus alignment**
- **subject isolation**

Overall Architecture Diagram



1. User & Presentation Layer

This layer represents the interaction interface for students and educators.

- Users submit academic queries through the frontend interface.
- The frontend (Streamlit/React UI) sends API requests to the backend server.
- The generated answers are displayed back to the user through the same interface.

Purpose: Provides an intuitive platform for subject selection, question input, and response visualization.

2. Application & Orchestration Layer

This layer controls the main application logic and workflow execution.

- The FastAPI backend receives user queries.
- User authentication is handled through Firebase.
- The backend forwards the query to the RAG pipeline.
- After processing, the backend returns the final generated answer to the frontend.

Purpose: Acts as the central coordinator connecting UI, retrieval, and AI generation components.

3. RAG Pipeline (Core Processing)

The RAG pipeline (implemented using LangChain) performs intelligent query processing.

Steps:

1. Constructs an augmented prompt from the user query
2. Converts the query into embeddings
3. Searches the FAISS vector database for relevant syllabus content
4. Retrieves contextual academic chunks
5. Sends context + query to the fine-tuned LLM
6. Generates syllabus-aligned response

Purpose: Ensures answers are grounded in academic knowledge instead of generic model memory.

4. AI & Data Layer

This layer contains the AI models and vector storage.

- **Fine-tuned LLM (HuggingFace + LoRA/PEFT): Generates academic answers**
- **Vector Database (FAISS): Stores embeddings of syllabus and notes**
- **The RAG pipeline retrieves relevant vectors from FAISS**
- **The LLM generates responses using retrieved context**

Purpose: Provides factual grounding and reduces hallucination.

5. Offline Data Preparation Layer

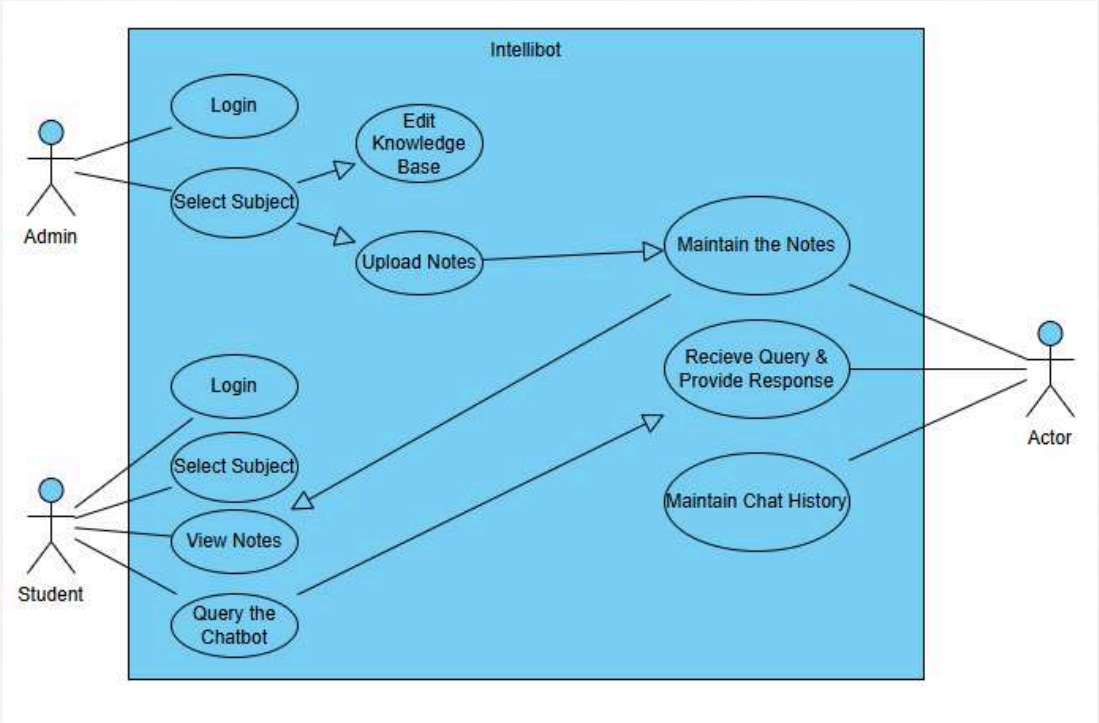
This layer prepares academic knowledge before runtime.

- **Knowledge base includes syllabus, notes, and PDFs**
- **Documents undergo chunking and embedding**
- **Embeddings are stored in FAISS vector database**
- **This prepared knowledge is later used by the RAG pipeline**

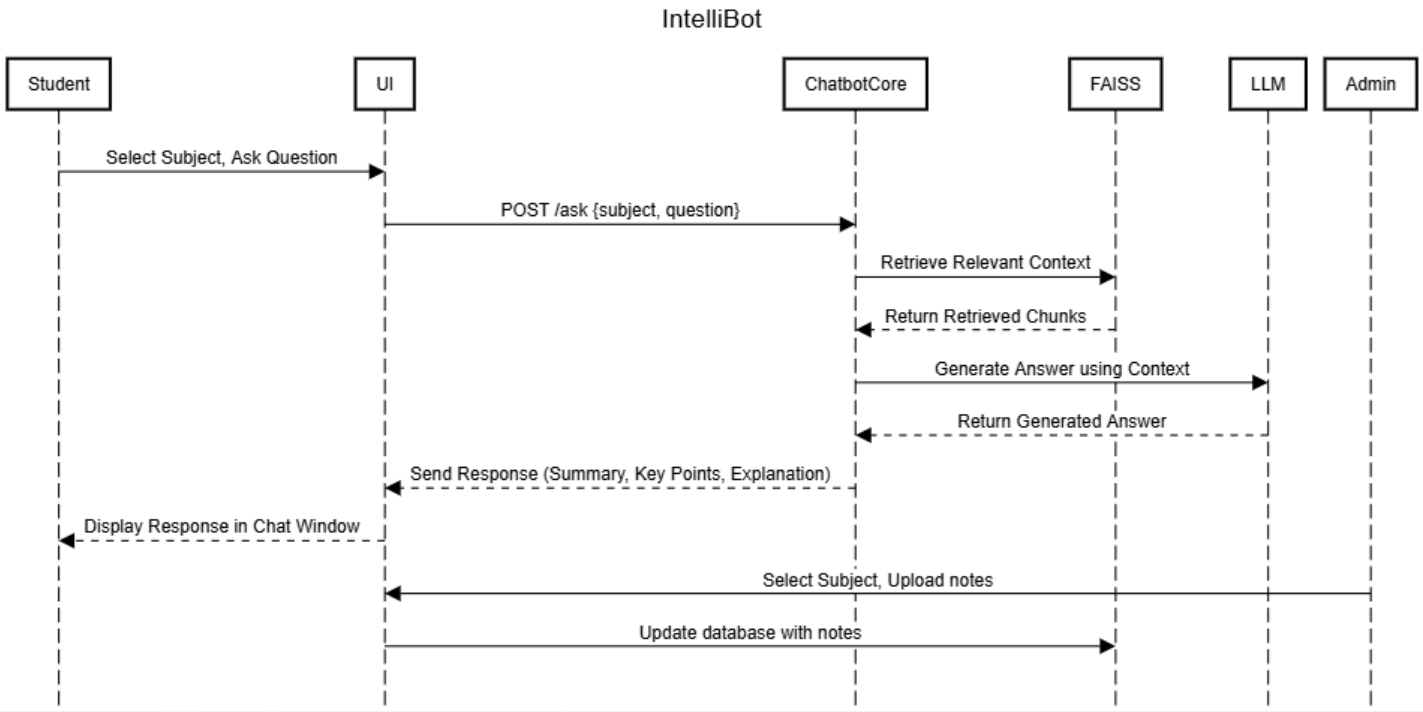
Purpose: Converts raw academic materials into searchable vector knowledge.

DESIGN DIAGRAMS

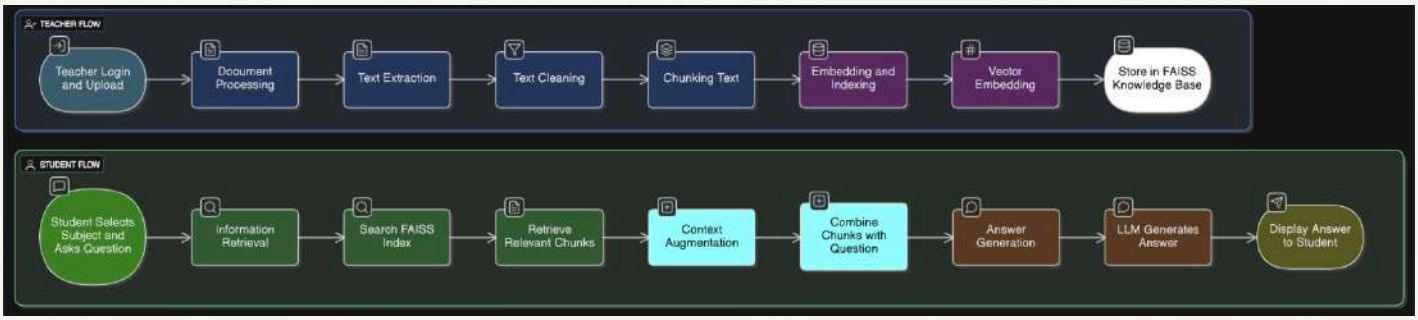
USE CASE DIAGRAMS



SEQUENCE DIAGRAM



DATAFLOW DIAGRAM

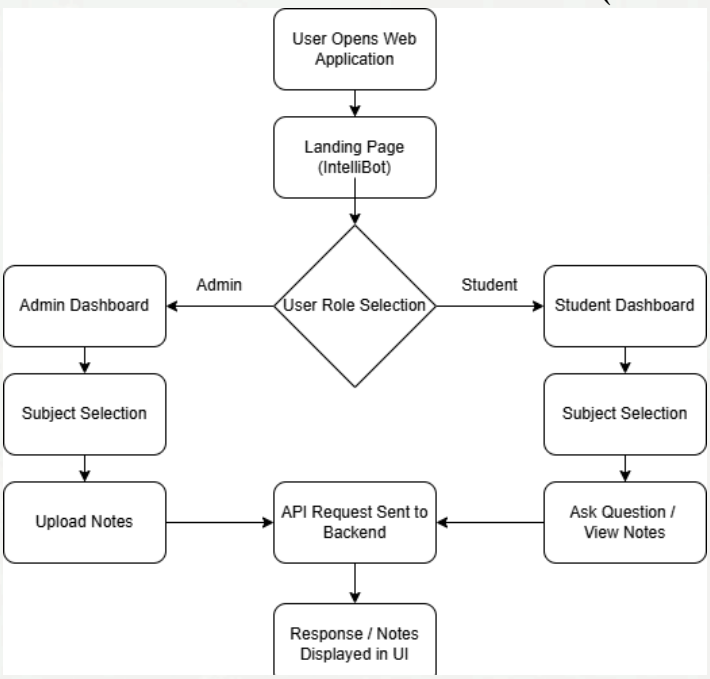


MODULE DESCRIPTION

User Interface Module (Frontend)

- Provides interaction layer between users and system
- Enables subject selection and question submission
- Supports teacher note upload and management
- Sends API requests to backend services
- Displays chatbot responses and learning materials

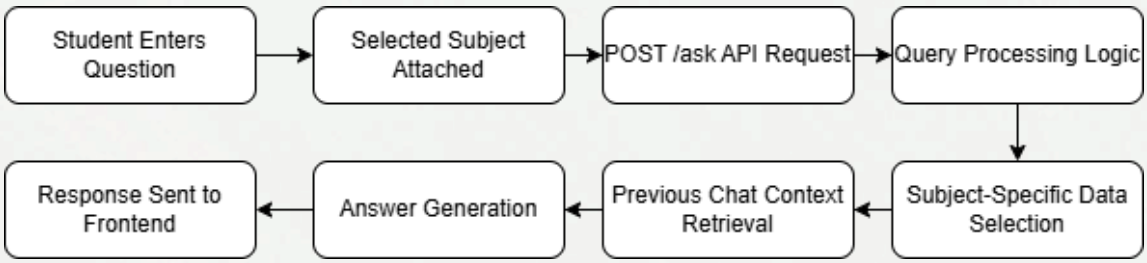
Module 1: User Interface Module (React)



Chatbot Core & Query Processing Module

- Receives and processes user queries
- Identifies selected subject and context
- Retrieves relevant academic content from vector database
- Constructs augmented prompt for LLM
- Generates syllabus-aligned and context-aware answers

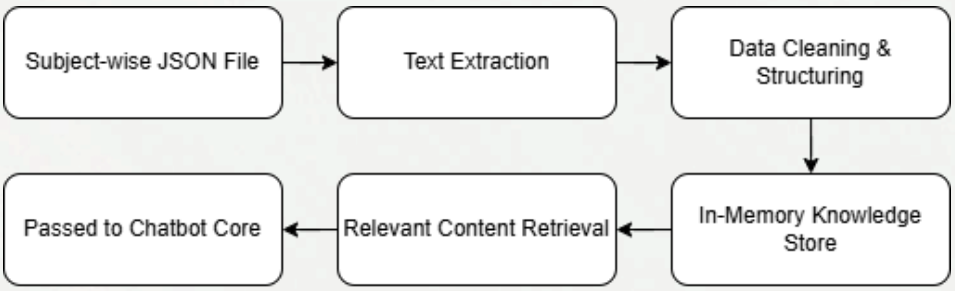
MODULE 2: CHATBOT CORE AND QUERY PROCESSING MODULE



Knowledge Base & Data Processing Module

- Converts syllabus, notes, and PDFs into structured data
- Performs text extraction, cleaning, and segmentation
- Generates embeddings for semantic search
- Stores processed vectors in FAISS database
- Supplies relevant knowledge during query processing

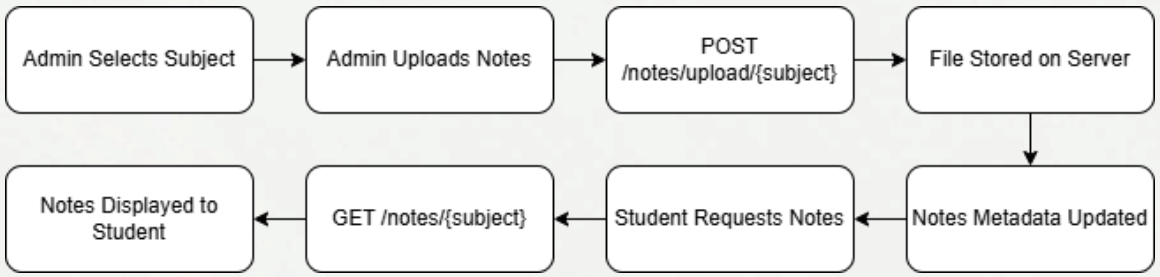
MODULE 3: KNOWLEDGE BASE AND DATA PROCESSING MODULE



Notes Management Module

- Allows teachers to upload, update, and delete notes
- Stores documents and updates metadata
- Automatically processes notes into knowledge base
- Makes uploaded materials searchable by chatbot
- Enables students to view and access notes

MODULE 4: NOTES MANAGEMENT MODULE

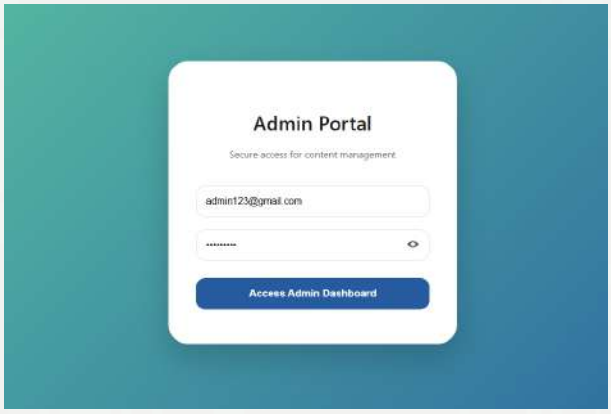


Results and Outputs

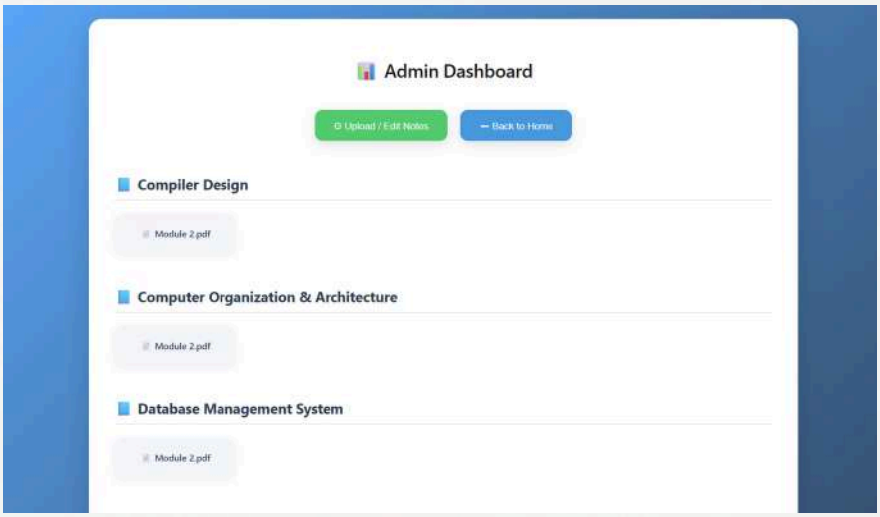
- **Fully functional multi-subject academic chatbot delivering accurate responses using semantic retrieval across OS, DBMS, COA, Python, and CD.**
- **Stable, production-ready React interface with subject-wise independent chat environments and seamless switching.**
- **Persistent chat memory implemented using Firebase, enabling saved conversations, auto-loading, and chat lifecycle management.**
- **Dynamic chat management features completed — create, rename (auto from first prompt), delete, and resume previous chats.**

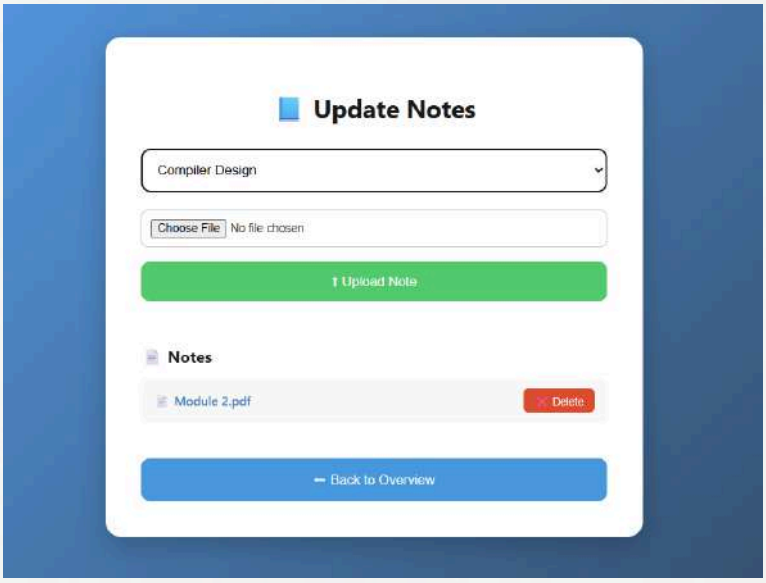
- Teachers can upload, update, and manage notes across subjects, with students accessing real-time updated academic resources.
- Role-based access control and secure data handling implemented for both students and instructors.
- Backend integrated and optimized (FastAPI + Firestore) for scalability, reliability, and multi-format academic document ingestion.
- End-to-end system testing, debugging, and UI refinement completed, making the platform deployment-ready.

OUTPUTS

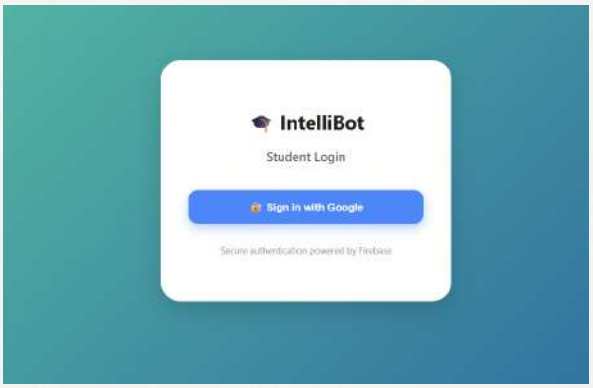


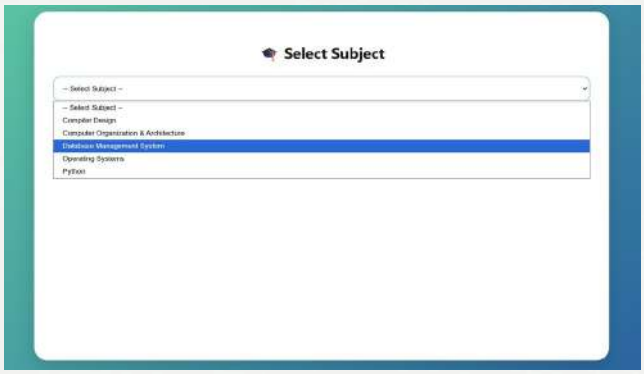
ADMIN SIDE


Admin Dashboard
Upload / Edit Notes
Back to Home
Compiler Design
Module 2.pdf
Computer Organization & Architecture
Module 2.pdf
Database Management System
Module 2.pdf

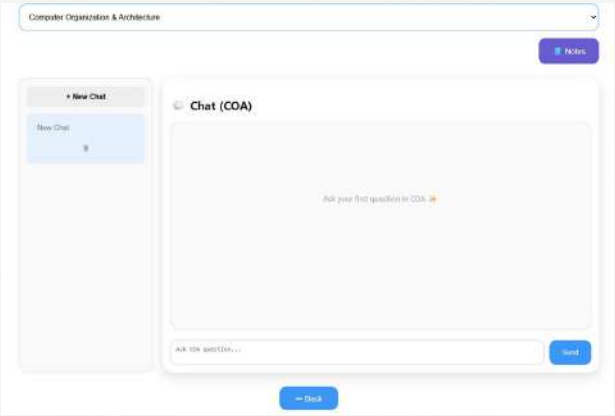

Update Notes
Compiler Design
Choose File No file chosen
Upload Note
Notes
Module 2.pdf
Delete
Back to Overview

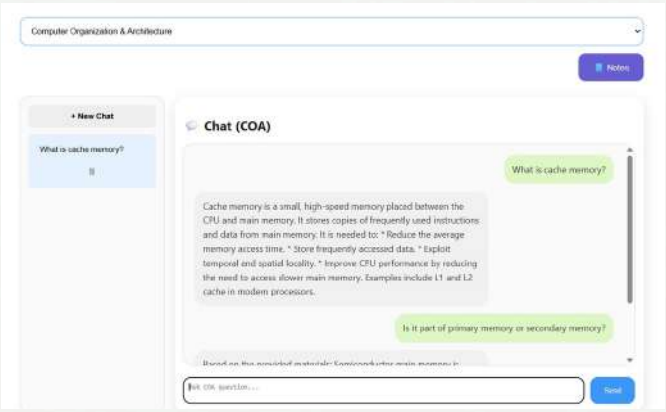
STUDENT SIDE



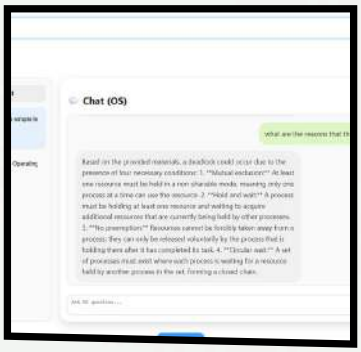


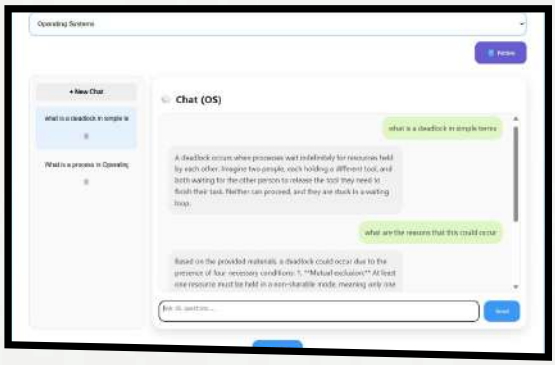
STUDENT SIDE

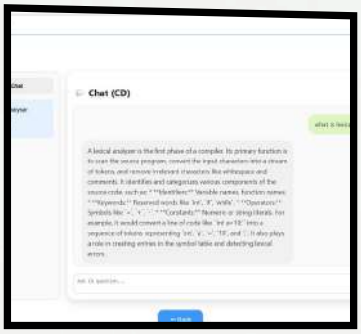


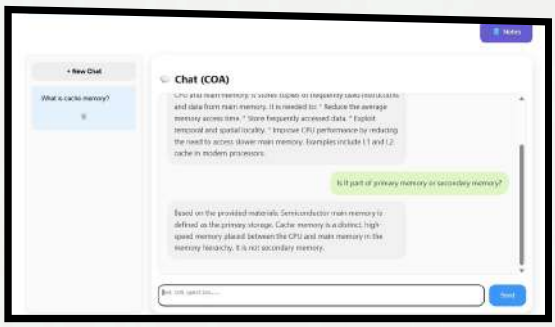


STUDENT SIDE

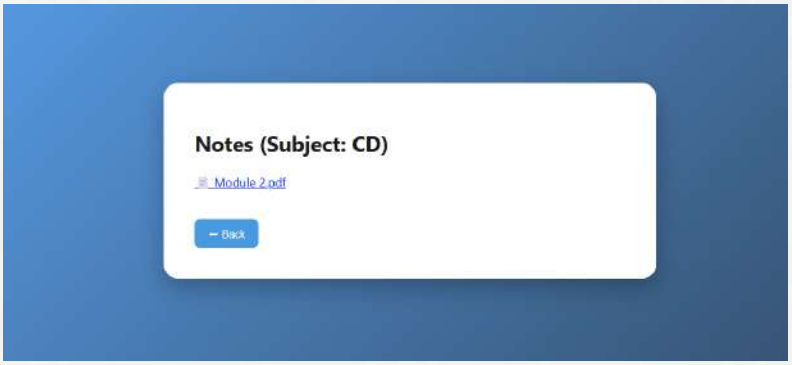








STUDENT SIDE



Assumptions

- Knowledge base prepared by teachers (Uploading the textbook)
- Subjects follow university syllabus
- Internet connectivity available
- Students ask academic questions only
- Uploaded documents are valid

Work breakdown and responsibilities

- Sheba Reji – Backend & RAG Pipeline
  - Designed and implemented the Fast API backend.
  - Integrated embeddings, FAISS retrieval, and prompt augmentation.
  - Ensured accurate, syllabus-aligned response generation.

- Nidha Rahiman Mothirapeedika – Frontend Development
  - Developed the React-based chat interface.
  - Handled API integration between React and Fast API.
  - Improved UI responsiveness and user interaction.

Work breakdown and responsibilities

- P S Ashna Parveen – Data Curation & Knowledge Base
  - Created and structured the Operating Systems JSON knowledge base.
  - Designed the multi-layered schema (summary, key points, explanations).
  - Verified content accuracy and syllabus alignment.

- Noel Mathachan Mampilly – System Integration & Documentation
  - Implemented overall system architecture and module-wise workflows.
  - Prepared diagrams, project documentation, and evaluation materials.
  - Coordinated testing and presentation preparation.

Hardware and Software Requirements

Hardware

- 8GB RAM minimum
- Intel i5 or above
- Internet

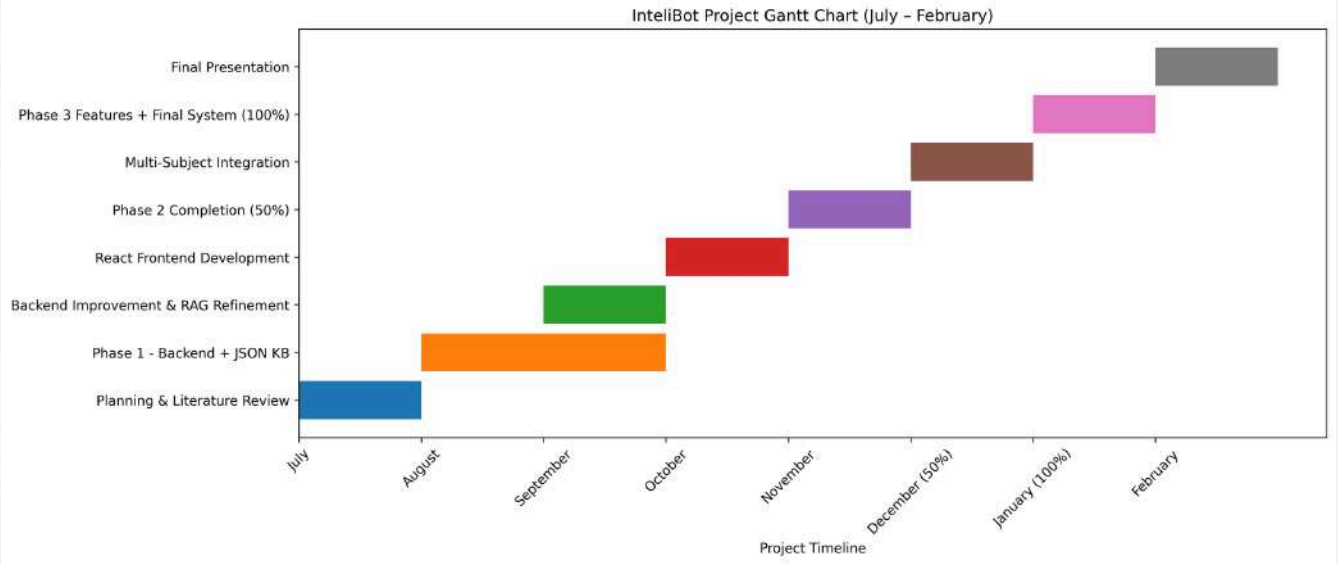
Datasets

- Subject-wise curated JSON
- Teacher notes (PDF, PPT, DOCX)

Software

- React
- FastAPI
- Python
- LangChain
- FAISS
- Firebase
- HuggingFace

Gantt Chart



Risks and Challenges

- **Hallucination control: Ensuring the AI model does not generate incorrect or fabricated information while answering queries.**
- **Knowledge base quality: The accuracy and reliability of the chatbot depend on how well the academic materials and datasets are curated.**
- **Multi-subject scaling: Expanding the system to support multiple subjects while maintaining efficient retrieval and response accuracy.**

Risks and Challenges

- Performance latency: The delay that occurs while retrieving data from the database and generating responses from the AI model.
- Data consistency: Maintaining uniform and updated information across all knowledge sources and system modules.
- Model dependency: The system's performance and accuracy depend heavily on the capabilities of the underlying AI model.

Future Implementation

- Teachers will be able to upload subject syllabus and textbooks directly into the system.
- The uploaded documents will be automatically processed and converted into structured JSON files to build a subject-wise knowledge base.
- The system will support adding more subjects to expand the knowledge base over time.
- The system can be expanded to include more subjects and courses from different departments, making it a scalable academic assistant.

Conclusion

- IntelliBot successfully implements a RAG-based academic learning assistant with multi-subject coverage.
- Migration to a React-based frontend improves scalability, modularity, and maintainability.
- Subject-wise chat separation enhances contextual accuracy and user experience.
- Integration of persistent memory and dynamic document ingestion strengthens system functionality.
- The current implementation provides a strong technical foundation for future optimization, evaluation, and research contributions.

References

1. Zawacki-Richter O., Marin V., Bond M., Gouverneur F. (2019). Systematic Review of Research on Artificial Intelligence Applications in Higher Education. https://doi.org/10.1186/s41239-019-0171-0
2. A. Sharma, R. Kumar, and S. Patel, "Applications of large language models in personalized learning: A survey," Computers & Education: Artificial Intelligence, vol. 6, 100200, 2025.
3. Kasneci E., Sessler K., Küchemann S., et al. (2023). ChatGPT for Good? On Opportunities and Challenges of Large Language Models for Education. https://doi.org/10.1007/s00146-023-01659-7
4. E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," in Proc. Int. Conf. Learning Representations (ICLR), 2022.
5. Lewis P., Perez E., Piktus A., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. https://arxiv.org/abs/2005.11401

6. Klimova B., et al. (2025). Artificial Intelligence Chatbots in Education: A Systematic Review. https://pmc.ncbi.nlm.nih.gov/articles/PMC11830699/
7. Devlin J., Chang M.W., Lee K., Toutanova K. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. https://arxiv.org/abs/1810.04805
8. Brown A., Roman M., Devereux B. (2025). A Systematic Literature Review of Retrieval-Augmented Generation: Techniques, Metrics and Challenges. https://arxiv.org/abs/2508.06401
9. Ji Z., Lee N., Frieske R., et al. (2023). Survey of Hallucination in Natural Language Generation. https://arxiv.org/abs/2302.03629
10. FAISS: Facebook AI Similarity Search Library. https://github.com/facebookresearch/faiss

11. Radford A., et al. (2019).
Language Models are Unsupervised Multitask Learners.
https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf

12. Brown T., et al. (2020).
Language Models are Few-Shot Learners.
https://arxiv.org/abs/2005.14165

13. Raffel C., et al. (2020).
Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer (T5).
https://arxiv.org/abs/1910.10683

The End

THANK YOU FOR LISTENING

Appendix B: Vision, Mission, Programme Outcomes and Course Outcomes

Vision, Mission, Programme Outcomes and Course Outcomes

Institute Vision

To evolve into a premier technological institution, moulding eminent professionals with creative minds, innovative ideas and sound practical skill, and to shape a future where technology works for the enrichment of mankind.

Institute Mission

To impart state-of-the-art knowledge to individuals in various technological disciplines and to inculcate in them a high degree of social consciousness and human values, thereby enabling them to face the challenges of life with courage and conviction.

Department Vision

To become a centre of excellence in Computer Science and Engineering, moulding professionals catering to the research and professional needs of national and international organizations.

Department Mission

To inspire and nurture students, with up-to-date knowledge in Computer Science and Engineering, ethics, team spirit, leadership abilities, innovation and creativity to come out with solutions meeting societal needs.

Programme Outcomes (PO)

Engineering Graduates will be able to:

1. Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop solutions for complex engineering problems.

2. Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development.

3. Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for public health and safety, whole-life cost, net zero carbon, culture, society and environment as required.

4. Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis and interpretation of data to provide valid conclusions.

5. Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering and IT tools, including prediction and modelling, recognizing their limitations to solve complex engineering problems.

6. The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment.

7. Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national and international laws.

8. Individual and Collaborative Team Work: Function effectively as an individual, and as a member or leader in diverse or multidisciplinary teams.

9. Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, and make effective presentations considering cultural, language and learning differences.

10. Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work as a member and leader in a team, and to manage projects in multidisciplinary environments.

11. Life-Long Learning: Recognize the need for, and have the preparation and ability for (i) independent and life-long learning, (ii) adaptability to new and emerging technologies, and (iii) critical thinking in the broadest context of technological change.

Programme Specific Outcomes (PSO)

A graduate of the Computer Science and Engineering Program will demonstrate:

PSO1: Computer Science Specific Skills
The ability to identify, analyze and design solutions for complex engineering problems in multidisciplinary areas by understanding the core principles and concepts of computer science and thereby engage in national grand challenges.

PSO2: Programming and Software Development Skills
The ability to acquire programming efficiency by designing algorithms and applying standard practices in software project development to deliver quality software products meeting the demands of the industry.

PSO3: Professional Skills
The ability to apply the fundamentals of computer science in competitive research and to develop innovative products to meet the societal needs thereby evolving as an eminent researcher and entrepreneur.

Course Outcomes (CO)

Course Outcome 1: Model and solve real world problems by applying knowledge across domains (Cognitive knowledge level: Apply).

Course Outcome 2: Develop products, processes or technologies for sustainable and

socially relevant applications (Cognitive knowledge level: Apply).

Course Outcome 3: Function effectively as an individual and as a leader in diverse teams and to comprehend and execute designated tasks (Cognitive knowledge level: Apply).

Course Outcome 4: Plan and execute tasks utilizing available resources within timelines, following ethical and professional norms (Cognitive knowledge level: Apply).

Course Outcome 5: Identify technology/research gaps and propose innovative/creative solutions (Cognitive knowledge level: Analyze).

Course Outcome 6: Organize and communicate technical and scientific findings effectively in written and oral forms (Cognitive knowledge level: Apply).

Appendix C: CO-PO-PSO Mapping

COURSE OUTCOMES:

SNO	DESCRIPTION	BLOOM'S TAXONOMY LEVEL
1	Model and solve real world problems by applying knowledge across domains.	Apply
2	Develop products, processes or technologies for sustainable and socially relevant application.	Apply
3	Function effectively as an individual and as a leader in diverse teams and to comprehend and execute designated tasks.	Apply
4	Plan and execute tasks utilizing available resources within timelines, following ethical and professional norms.	Apply
5	Identify technology/research gaps and propose innovative/creative solutions.	Analyze
6	Organize and communicate technical and scientific findings effectively in written and oral forms.	Apply

MAPPING COURSE OUTCOMES (COs) – PROGRAM OUTCOMES (POs) AND PROGRAM SPECIFIC OUTCOMES (PSOs)

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
CO1	2	2	2	1		2	1	1	1	1	1	2	2	2
CO2	2	2	2		2	3	2	1	1	1	1	2	2	
CO3							3	2	2	1				3
CO4		1	2		3	2	3	2	2	3	1		2	2
CO5	2	3	3	1	2				1	3	3			
CO6			2			2	2	2	3	1	1			3

JUSTIFICATIONS FOR CO-PO MAPPING

MAPPING	LEVEL	JUSTIFICATION
101003/ CS822U.CO1- PO1	Medium	Applies engineering knowledge, mathematics, and computing fundamentals to model and solve real-world problems.
101003/ CS822U.CO1- PO2	Medium	Requires identification, formulation and analysis of engineering problems before developing solutions.
101003/ CS822U.CO1- PO3	Medium	Involves designing systems or solutions to address identified engineering needs.
101003/ CS822U.CO1- PO4	Low	Limited investigation or experimentation may be required to validate the developed models or solutions.
101003/ CS822U.CO1- PO6	Medium	Solutions developed may influence societal, environmental, or sustainability aspects of engineering applications.
101003/ CS822U.CO1- PO7	Medium	Encourages consideration of ethical and professional responsibilities while developing solutions.
101003/ CS822U.CO1- PO8	Low	Some level of teamwork may be required while working on modeling or project tasks.
101003/ CS822U.CO1- PO9	Low	Requires communication of results and findings through reports or presentations.
101003/ CS822U.CO1- PO10	Low	Involves basic planning and execution of tasks while implementing solutions.
101003/ CS822U.CO1- PO11	Low	Encourages independent learning and adapting new knowledge to solve real-world problems.
101003/ CS822U.CO1- PSO1	Medium	Uses core computer science principles to analyze problems and design appropriate technical solutions.

101003/ CS822U.CO1- PSO2	Medium	Involves algorithm design and application of software development practices to implement solutions.
101003/ CS822U.CO1- PSO3	Medium	Applies computer science fundamentals in research or innovative problem solving for societal needs.
101003/ CS822U.CO2- PO1	Medium	Utilizes engineering knowledge to design and develop products or systems.
101003/ CS822U.CO2- PO2	Medium	Requires analysis of user needs and technological feasibility before development.
101003/ CS822U.CO2- PO3	Medium	Focuses on design and development of solutions addressing real-world requirements.
101003/ CS822U.CO2- PO5	Medium	Requires use of modern engineering and IT tools for system or product development.
101003/ CS822U.CO2- PO6	High	Emphasizes sustainability and societal impact during solution development.
101003/ CS822U.CO2- PO7	Medium	Ethical and professional responsibilities are considered when developing socially relevant technologies.
101003/ CS822U.CO2- PO8	Low	Development activities may require collaboration among team members.
101003/ CS822U.CO2- PO9	Low	Involves communicating design ideas and development outcomes.
101003/ CS822U.CO2- PO10	Low	Requires basic planning and coordination during development tasks.
101003/ CS822U.CO2- PO11	Low	Encourages continuous learning of emerging technologies while building solutions.
101003/ CS822U.CO2- PSO1	Medium	Applies computer science fundamentals to design technically feasible solutions.
101003/ CS822U.CO2- PSO2	Medium	Uses programming and software development practices to implement solutions.

101003/ CS822U.CO3- PO8	High	Strongly relates to functioning effectively as a member or leader in multidisciplinary teams.
101003/ CS822U.CO3- PO9	Medium	Requires effective communication among team members during collaboration.
101003/ CS822U.CO3- PO10	Medium	Team leadership involves project coordination and task management.
101003/ CS822U.CO3- PO11	Low	Team interaction supports learning from peers and adapting new approaches.
101003/ CS822U.CO3- PSO3	High	Professional collaboration and teamwork support innovative computer science solutions.
101003/ CS822U.CO4- PO2	Low	Requires analysis and planning before executing project tasks.
101003/ CS822U.CO4- PO3	Medium	Involves designing and implementing project solutions based on planned activities.
101003/ CS822U.CO4- PO5	High	Requires effective use of modern tools and resources during project execution.
101003/ CS822U.CO4- PO6	Medium	Considers societal and environmental impact while implementing solutions.
101003/ CS822U.CO4- PO7	High	Emphasizes adherence to ethical practices and professional responsibilities.
101003/ CS822U.CO4- PO8	Medium	Task execution may involve teamwork and coordination among team members.
101003/ CS822U.CO4- PO9	Medium	Requires communication of progress, outcomes and documentation.
101003/ CS822U.CO4- PO10	High	Focuses on project management including scheduling, resource allocation and task execution.
101003/ CS822U.CO4- PO11	Low	Encourages adaptability and continuous learning during project implementation.

101003/ CS822U.CO4- PSO2	Medium	Uses software development practices to execute and manage project tasks.
101003/ CS822U.CO4- PSO3	Medium	Applies computer science knowledge in practical project development environments.
101003/ CS822U.CO5- PO1	Medium	Uses engineering knowledge to understand technological limitations and opportunities.
101003/ CS822U.CO5- PO2	High	Requires deep problem analysis and identification of research gaps.
101003/ CS822U.CO5- PO3	High	Proposes innovative design solutions to address identified gaps.
101003/ CS822U.CO5- PO4	Low	May involve limited investigation or validation of proposed ideas.
101003/ CS822U.CO5- PO5	Medium	Uses appropriate tools and techniques to analyze and develop innovative solutions.
101003/ CS822U.CO5- PO10	Low	Requires minimal planning while proposing new solution approaches.
101003/ CS822U.CO5- PO11	High	Strongly promotes independent learning and exploration of emerging technologies.
101003/ CS822U.CO5- PSO1	High	Applies computer science fundamentals to identify and solve complex technical problems.
101003/ CS822U.CO6- PO3	Medium	Requires presentation of design concepts and developed solutions.
101003/ CS822U.CO6- PO6	Medium	Communication may include discussion of societal or environmental implications.
101003/ CS822U.CO6- PO7	Medium	Requires responsible and ethical communication of technical results.
101003/ CS822U.CO6- PO8	Medium	Communication supports collaboration within team environments.

101003/ CS822U.CO6- PO9	High	Strongly relates to effective written and oral communication of technical findings.
101003/ CS822U.CO6- PO10	Low	Requires structured reporting and documentation in project management contexts.
101003/ CS822U.CO6- PO11	Low	Encourages continuous improvement in communication and presentation skills.
101003/ CS822U.CO6- PSO3	High	Communication of innovative computer science solutions and research outcomes.